LAW: Unifying Language, Action, and World Modeling for Agentic Physical Intelligence

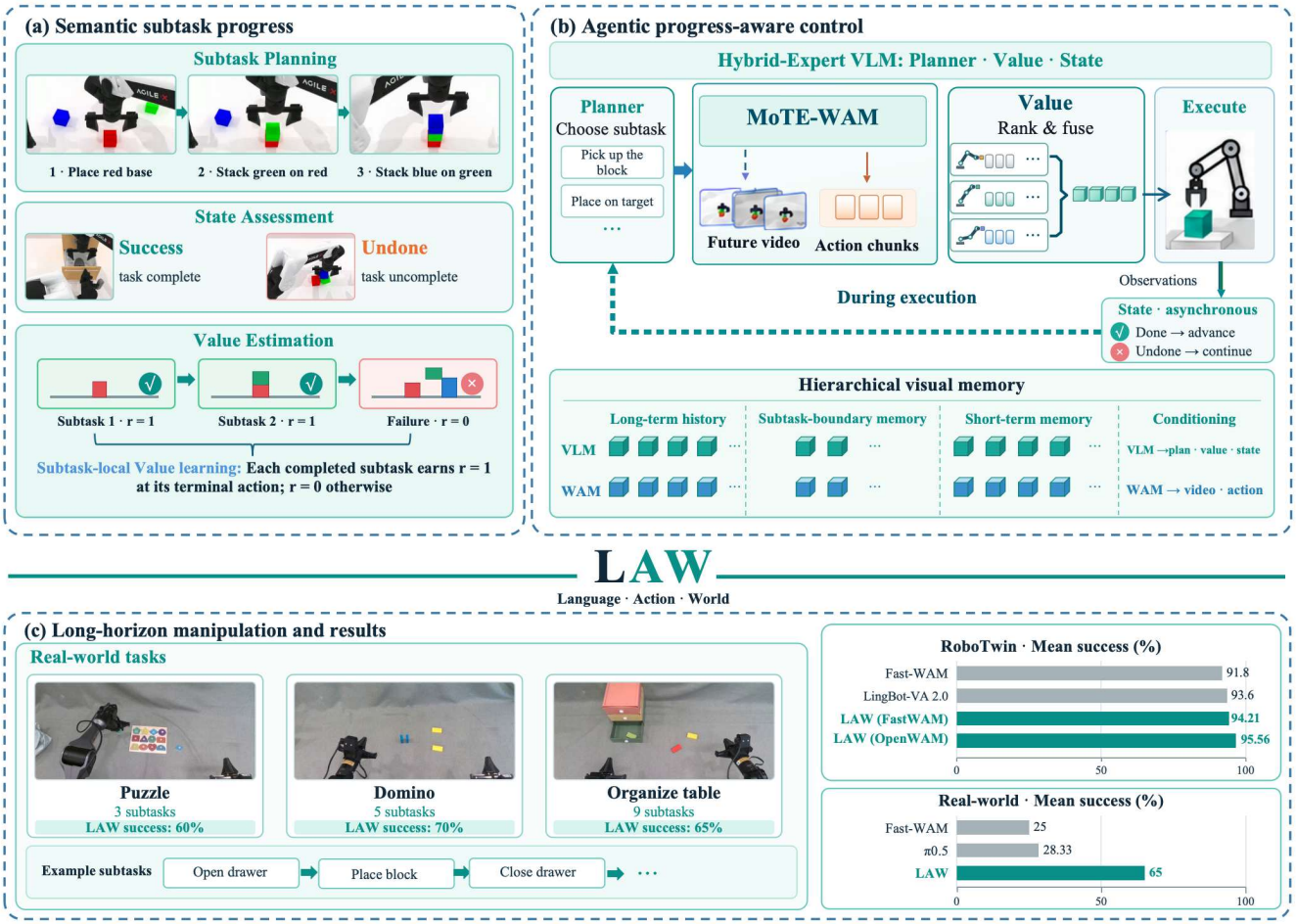


Fig. 1. Overview of LAW. (a) Semantic subtask progress encompasses subtask planning, subtask completion assessment, and value estimation. (b) Progress-aware closed-loop control with hybrid-expert VLM, MoTE-WAM, and dual-stream hierarchical visual memory. (c) Evaluation on RoboTwin and long-horizon real-world manipulation tasks.

Abstract—World-action models jointly predict actions and future world states, but long-horizon manipulation further requires language reasoning to plan subtask objectives, assess execution progress, and guide action selection. We present LAW, a closed-loop VLM–WAM framework that unifies language reasoning, action generation, and world modeling through semantic subtask progress. Within the LAW agent, a hybrid-expert vision-language model plans the remaining subtasks, estimates the values of and fuses actions proposed by a world-action model, and asynchronously assesses execution status to support continued execution and subtask transitions. Within the world-action model, Mixture-of-Transformer Experts integrates modality-specific and shared computation for joint video–action modeling. Dual-stream hierarchical visual memory maintains temporal context for both models, while subtask-local value learning aligns action evaluation with semantic objectives and preserves supervision from successful prefixes of failed trajectories. LAW achieves 94.21% average success across 50 RoboTwin tasks. Replacing the WAM backbone with OpenWAM-α further raises the average success rate to 95.56%, establishing a new open-source state of the art on RoboTwin. Despite having only 2B parameters, our VLM also outperforms variants using Embodied-R1.5 and GPT-5.5 High. LAW further leads every RoboDojo capability dimension within the group of methods without additional robot-domain or embodied-interaction pretraining. On three long-horizon real-robot tasks, LAW achieves 65% average success, compared with 28.33% for $\pi_{0.5}$ and 25% for Fast-WAM. The project website and code are available at xxx.

1 Introduction

Long-horizon manipulation is a sequence of semantic subtasks, not merely a longer sequence of motor commands. After reaching for an object, a robot must decide whether the grasp actually succeeded; after a successful grasp, it must recognize that the task has entered a new stage; before executing the next motion, it must choose among several physically plausible actions according to the current objective [1], [2]. Errors in any of these decisions compound over time. Consequently, local action quality alone does not determine task success. A capable agent must maintain a persistent account of task progress, connect that account to action generation, and revise its decisions when execution departs from expectation [3], [4].

Vision-language-action policies connect semantic representations to control [5]–[7], while world-action models learn action-conditioned scene evolution and executable behavior [8]–[11]. These models substantially improve action generation, but long-horizon deployment exposes a complementary systems problem: reasoning and control must remain aligned as the task evolves. A language plan generated once at the beginning cannot determine whether its intermediate steps were completed [2], and an action generator conditioned only on the current observation may not know which visual changes matter to the larger goal [12]. Placing a planner before a policy therefore leaves three interfaces underspecified: how execution progress updates the plan, how candidate actions are judged against the active semantic objective, and how relevant history is retained without overwhelming either model.

Our central premise is that semantic subtask progress should serve as the temporal interface between language reasoning and world-action modeling. Subtasks provide more than a decomposition of the instruction: they define when the agent should advance its plan, which outcomes a candidate action should optimize, and which observations should remain salient in memory. This view also reveals that the required computations occur at different rates. State assessment should run continuously without blocking control, candidate evaluation should occur whenever the WAM proposes a new action chunk, and planning should operate at the semantic task scale rather than at every control step. The challenge is thus not to invoke a larger reasoner at every step, but to organize specialized reasoning and control processes around a shared notion of semantic subtask progress.

We instantiate this principle in LAW, a closed-loop VLM–WAM framework that unifies language reasoning, action generation, and world modeling through semantic subtask progress, as summarized in Fig. 1. The LAW agent uses a hybrid-expert VLM with role-selected planner, state, and value experts on a shared frozen backbone. The planner predicts the remaining semantic subtasks; conditioned on the active subtask, the WAM generates diverse action chunks; the value expert ranks and fuses them; and execution status is assessed asynchronously to determine subtask completion. Once the active subtask is completed, the planner predicts the next subtask and the agent switches to it; otherwise, the agent continues executing the current subtask. The LAW agent coordinates planning, action selection, and execution monitoring at complementary frequencies, using progress feedback to guide continued execution and subtask transitions.

To strengthen joint world and action modeling, we introduce Mixture-of-Transformer Experts (MoTE) into the WAM, yielding MoTE-WAM. MoT projects video and action features into a common attention space to exchange cross-modal information, but their subsequent transformations remain modality-specific. MoTE introduces shared experts to further transform these cross-modal representations, alongside video-private and action-private experts that preserve modality-specific computation. The shared experts learn common temporal and physical structure across predicted scene dynamics and robot actions, strengthening video–action joint modeling beyond information exchange through attention alone. Long-horizon execution also requires memory aligned with semantic subtask progress. Separate hierarchical memory streams for the VLM and WAM retain recent observations and full-resolution states at subtask boundaries, while compressing older history to preserve relevant context across subtasks. To better exploit failed trajectories when training the value expert, we introduce subtask-local value learning, which terminates returns at semantic boundaries. This preserves positive supervision from successfully completed subtasks within failed episodes, rather than assigning zero value to all their actions.

We evaluate LAW on 50 RoboTwin [13] tasks, the RoboDojo [14] capability benchmark, and three long-horizon real-robot tasks containing up to nine semantic subtasks. LAW achieves 94.21% average success on RoboTwin, 2.4 percentage points above Fast-WAM, without robot-data pretraining. Replacing the WAM backbone with OpenWAM-α further raises the average success rate to 95.56%, establishing a new open-source state of the art on RoboTwin. Despite having only 2B parameters, our VLM outperforms variants using Embodied-R1.5 and GPT-5.5 High within the same LAW framework. On RoboDojo, LAW achieves the best result in every capability dimension among methods without additional robot-domain or embodied-interaction pretraining. On the three long-horizon real-robot tasks, LAW achieves 65% average success, compared with 28.33% for $\pi_{0.5}$ and 25% for Fast-WAM. We also conduct extensive ablation studies to validate the effectiveness of each component. Our contributions are:

- We introduce LAW, a closed-loop VLM–WAM framework that unifies language reasoning, action generation, and world modeling through semantic subtask progress. The LAW agent coordinates subtask planning, asynchronous completion assessment, and value-guided action fusion to support continuous long-horizon execution.
- We develop three complementary mechanisms for progress-aware control: Mixture-of-Transformer Experts strengthens joint video–action modeling through shared and modality-specific experts; dual-stream hierarchical memory preserves task-relevant history and subtask-boundary observations; and subtask-local value learning retains positive supervision from completed stages of failed trajectories.
- We demonstrate LAW's effectiveness on RoboTwin,

RoboDojo, and long-horizon real-world manipulation through extensive experiments and ablation studies. Integrating OpenWAM-α achieves 95.56% average success on RoboTwin, establishing a new open-source state of the art and demonstrating LAW's extensibility to stronger pretrained WAMs.

2 Related Work

2.1 Vision-Language-Action Models for Robot Control

Pretrained vision-language models (VLMs) provide semantic representations that vision-language-action (VLA) policies connect to executable control [5]–[7], [15], [16], supported by heterogeneous robot datasets and generalist pretraining [17], [18]. RT-2 and OpenVLA represent actions as tokens, whereas π_0 couples a VLM with a flow-matching expert for continuous action generation [5]–[7]. Beyond action parameterization, CoT-VLA predicts intermediate visual subgoals [19], while VEGA aligns visual-encoder outputs with geometry-aware features before vision-language fusion [20]. These approaches strengthen semantic transfer and spatial grounding, motivating complementary mechanisms for coordinating instructions, execution progress, and action selection over extended tasks.

Language-guided systems connect reasoning to execution through affordance-based skill selection, environmental feedback, action hierarchies, and spatial value maps [1], [2], [21], [22]. SayCan grounds language plans in learned affordances, whereas VoxPoser translates language-specified objectives and constraints into spatial maps for trajectory optimization [1], [22]. In navigation, NavCoT structures decisions through future imagination and observation matching [23], while Memoir retrieves environmental and behavioral experience using imagined states [24]. Although navigation differs from continuous manipulation, these studies illustrate how intermediate reasoning and selective memory access support sequential decisions. For long-horizon robot policies, the corresponding challenge is to retain task-relevant history and execution feedback when semantic stages extend beyond individual action chunks [4], [12].

2.2 World Action Models and Video-Based Policy

World models support control through action-conditioned visual prediction or latent dynamics [25]–[29], while video generators also enable synthetic robot training trajectories [30]. Visual Foresight optimizes actions through predicted observations, whereas Dreamer learns behavior through latent imagination, providing two complementary uses of predictive dynamics [25], [26]. Recent world-action models (WAMs) connect future visual modeling with executable action generation through joint diffusion or future-latent alignment [8]–[11], [31]–[33]. Their designs differ in how predictive information is used during deployment: LingBot-VA combines causal video–action modeling with observation feedback and asynchronous execution [10], whereas Fast-WAM retains video co-training but omits test-time future prediction [11]. WAM4D further introduces geometric supervision through spatial register tokens and modality-specific causal attention [34]. These developments distinguish the benefits of predictive supervision from the computational requirements of generating future observations online.

Reliable world-model-based control also requires faithful action consequences and informative evaluation signals. WorldEcho diagnoses failures to follow off-expert actions, and WorldSync improves action-conditioned generation through broader coverage and explicit action-effect alignment [35]. Cosmos Policy jointly generates actions, future states, and values for planning [36]; MV-WAM integrates progress-value estimation with value-guided rollback [37]; and SafeDojo optimizes VLA policies using imagined trajectories with separate task-reward and safety-cost estimates [38]. LAW builds on these ingredients by coordinating semantic planning, asynchronous state assessment, and value-guided WAM candidate selection, supported by hierarchical visual memory. Its modality-specific and shared experts strengthen video–action interaction, while separate hierarchical memory streams preserve temporal context for language-level reasoning and continuous robot action generation. Unlike the trajectory-level returns used by MV-WAM [37], LAW terminates value targets at semantic subtask boundaries, preserving supervision from completed prefixes of unsuccessful episodes. This links local action evaluation to progress-aware action selection and planning grounded in semantic subtasks.

3 Method

3.1 LAW Framework Overview

Figure 2 provides an overview of the LAW agent's architecture and loop. The LAW agent factorizes long-horizon control into a semantic subtask progress loop and a short-horizon action loop. At control step t, semantic subtask progress is represented by an active-subtask index j_t in the current plan, while two memory streams, $\mathcal{M}_t^{\mathrm{V}}$ and $\mathcal{M}_t^{\mathrm{W}}$, provide task-relevant history to the VLM and WAM, respectively. The active subtask connects the two loops: it specifies the objective for action generation and value estimation, while execution feedback determines whether that objective should be continued or advanced.

Given a global instruction g, current observation o_t, and VLM memory $\mathcal{M}_t^{\mathrm{V}}$, the planner predicts the remaining semantic subtask sequence

$$\mathcal{S}_t = \mathrm{VLM}_{\mathrm{plan}}(g, o_t, \mathcal{M}_t^{\mathrm{V}}) = (s_{j_t}, s_{j_t+1}, \ldots, s_J), \quad (1)$$

where s_{j_t} is the active subtask and s_J is the final subtask. The planner is invoked at episode initialization and subsequently at semantic decision events rather than at every control step. Conditioned on s_{j_t}, the current observation, and WAM memory, the WAM samples N candidate action chunks:

$$\{A_t^{(n)}\}_{n=1}^{N} = \mathrm{WAM}_{\mathrm{act}}(o_t, s_{j_t}, \mathcal{M}_t^{\mathrm{W}}; \epsilon_n). \quad (2)$$

Each candidate $A_t^{(n)} = \{a_{t:t+H-1}^{(n)}\}$ contains an action sequence of horizon $H = 32$, and ϵ_n is an independent noise initialization that induces proposal diversity. Following Fast-WAM [11], the video branch provides a joint video–action training signal but is not sampled to produce explicit future images at inference. With VLM context

$$x_t = (g, o_t, s_{j_t}, \mathcal{M}_t^{\mathrm{V}}), \quad (3)$$

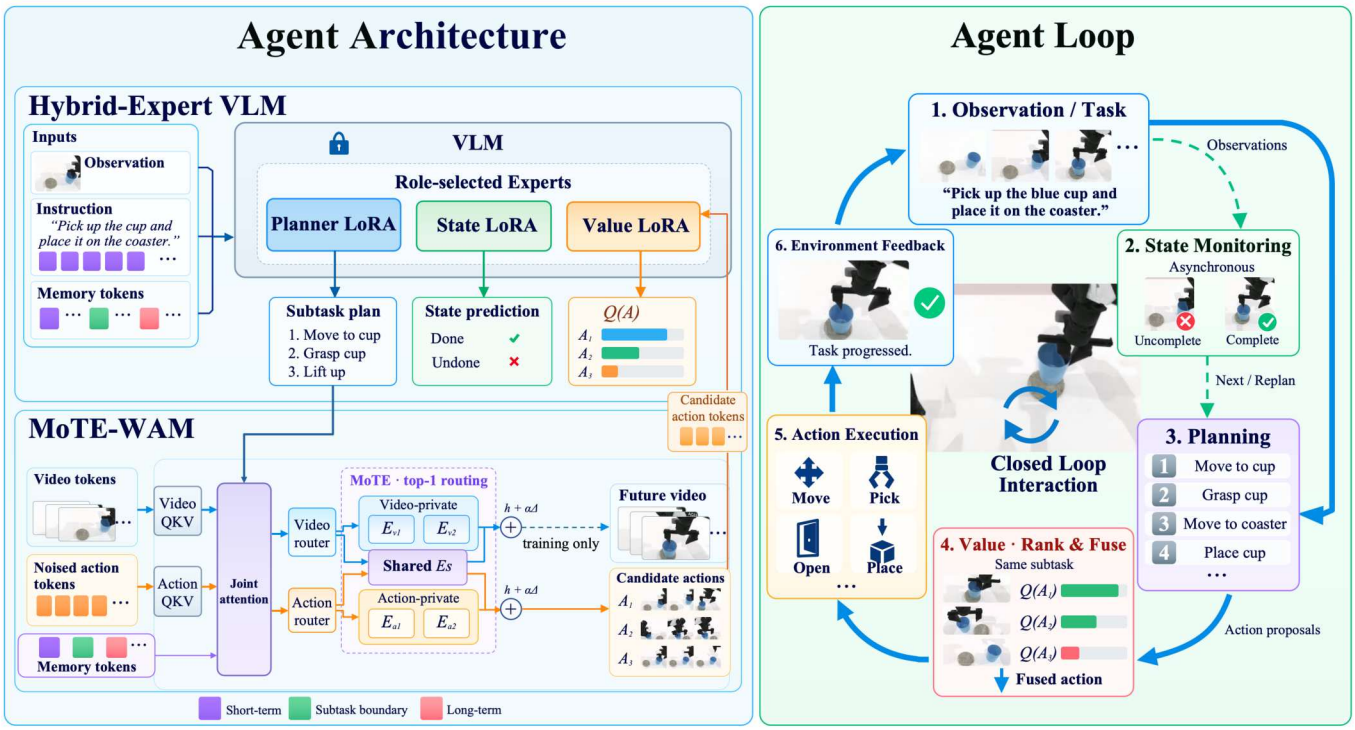


Fig. 2. LAW agent's architecture and loop.The hybrid-expert VLM uses planner, state, and value experts on a shared frozen backbone, while the MoTE-WAM is co-trained with video and action objectives and generates candidate action chunks at inference. Semantic subtask progress connects execution monitoring, subtask advancement, candidate ranking and fusion, action execution, and environment feedback in a closed loop.

the value expert scores each candidate and fuses the top-K proposals:

$$q_t^{(n)} = Q_\theta(x_t, A_t^{(n)}), \qquad \widehat{A}_t = \mathrm{Fuse}_{K,T}\left(\{A_t^{(n)}, q_t^{(n)}\}_{n=1}^{N}\right), \tag{4}$$

Here, Q_θ denotes the subtask-local value function implemented by the value expert, which estimates the expected return of a candidate under the active subtask. We use $K = 8$, and T is the fusion temperature. The robot executes a prefix of $\widehat{A}_t$ before the WAM generates new proposals from the next observation. Section 3.5 specifies the subtask-local target and the fusion operator.

In parallel with action execution, the state expert estimates whether the active subtask has completed:

$$\hat{d}_t = \mathrm{VLM}_{\mathrm{state}}(g, o_t, s_{j_t}, \mathcal{M}_t^{\mathrm{V}}). \tag{5}$$

The completion prediction is binarized at 0.5. In the current implementation, subtask completion determines the semantic control event

$$e_t = \begin{cases} \textsc{Advance}, & \hat{d}_t \geq 0.5, \\ \textsc{Continue}, & \text{otherwise.} \end{cases} \tag{6}$$

A CONTINUE event preserves j_t and the current plan. An ADVANCE event stores the transition observation as a boundary state and terminates the episode if the final subtask has completed. Otherwise, the planner predicts the remaining subtasks again using the updated observation and memory, and the LAW agent switches to the next predicted subtask. The new remaining sequence is indexed from $j_{t+1} = j_t + 1$, with its first element defining the next active subtask. Thus, semantic subtask completion changes the conditioning objective, memory state, and planning schedule of subsequent action generation. This feedback realizes the temporal interface described in the Introduction, while the following subsections detail the hybrid-expert VLM, MoTE-WAM, dual-stream memory, and subtask-local value learning.

Algorithm 1 summarizes the complete inference loop. State assessment runs concurrently with prefix execution. Each result is associated with its observation time and active subtask; the control loop reads the latest valid result without waiting for inference to finish and discards stale results. If no valid result is available, execution continues under the current subtask and plan.

3.2 Hybrid-Expert VLM for Progress-Aware Control

Hybrid-Expert VLM To support planning, execution monitoring, and action evaluation within a single VLM, we introduce a hybrid-expert architecture with task-specific LoRA adapters [39]. The planner, state, and value experts share a frozen VLM backbone, allowing them to reuse common visual-language representations while avoiding the cost of maintaining a separate full model for each control function. Each expert activates its own adapter and task-specific head, enabling specialization under distinct supervision objectives while isolating parameter updates and reducing interference across functions. Expert adapters are selected by control function rather than token-level routing.

The planner is trained with semantic subtask supervision. We construct ordered subtask annotations and temporal boundaries from the demonstrations, as illustrated in Fig. 3. Additional task decompositions and segmented

Algorithm 1 Progress-aware inference of the LAW agent

Require: Instruction g, initial observation o_0, candidate count N, top-K size K, temperature T

Initialize $t \leftarrow 0$, $\mathcal{M}_0^{\mathrm{V}} \leftarrow \emptyset$, and $\mathcal{M}_0^{\mathrm{W}} \leftarrow \emptyset$
$\mathcal{S}_0 \leftarrow \mathrm{VLM}_{\mathrm{plan}}(g, o_0, \mathcal{M}_0^{\mathrm{V}})$ and $j_0 \leftarrow 1$
while the task is not terminated **do**
  $\{A_t^{(n)}\}_{n=1}^{N} \leftarrow \mathrm{WAM}_{\mathrm{act}}(o_t, s_{j_t}, \mathcal{M}_t^{\mathrm{W}}; \epsilon_n)$
  $x_t \leftarrow (g, o_t, s_{j_t}, \mathcal{M}_t^{\mathrm{V}})$
  for $n = 1, \ldots, N$ **do**
    $q_t^{(n)} \leftarrow Q_\theta(x_t, A_t^{(n)})$
  end for
  $\widehat{A}_t \leftarrow \mathrm{Fuse}_{K,T}(\{A_t^{(n)}, q_t^{(n)}\}_{n=1}^{N})$
  Execute a prefix of $\widehat{A}_t$ while running $\mathrm{VLM}_{\mathrm{state}}$ asynchronously
  Receive o_{t+1} and update $\mathcal{M}_{t+1}^{\mathrm{V}}$ and $\mathcal{M}_{t+1}^{\mathrm{W}}$
  $j_{t+1} \leftarrow j_t$ and $\mathcal{S}_{t+1} \leftarrow \mathcal{S}_t$
  Read the latest available $\hat{d}$ without waiting; discard stale results
  if a valid result for s_{j_t} is available and $\hat{d} \geq 0.5$ **then**
    Store o_{t+1} as a semantic subtask-boundary observation
    if $s_{j_t} = s_J$ **then**
      terminate successfully
    else
      $j_{t+1} \leftarrow j_t + 1$
      $\mathcal{S}_{t+1} \leftarrow \mathrm{VLM}_{\mathrm{plan}}(g, o_{t+1}, \mathcal{M}_{t+1}^{\mathrm{V}})$
      Set $s_{j_{t+1}}$ to the first subtask of $\mathcal{S}_{t+1}$
    end if
  else
    $j_{t+1} \leftarrow j_t$
  end if
  $t \leftarrow t + 1$
end while

examples are provided in Appendix C. Given the global instruction, visual observation, and execution memory, the planner is trained to generate the remaining subtask sequence autoregressively:

$$\mathcal{L}_{\mathrm{plan}} = -\sum_{i=1}^{|\mathcal{S}|} \log p_\theta\Big(s_i \mid g, o_t, \mathcal{M}_t^{\mathrm{V}}, s_{<i}\Big). \tag{7}$$

The state expert is trained with subtask completion supervision and an auxiliary failure-prediction objective to improve completion assessment. For each annotated subtask, we compute a normalized progress score from the robot-state changes in translation, rotation, and gripper motion, and mark a frame as completed when the score exceeds 0.995. Failure trajectories are additionally reviewed to identify the first visually unambiguous manipulation error, together with its semantic stage and visual evidence. During training, dedicated completion and failure query tokens provide complementary supervision: the auxiliary failure head encourages the shared representation to distinguish unsuccessful execution from completed subtasks. The training objective is

$$\mathcal{L}_{\mathrm{state}} = \lambda_d \,\mathrm{BCE}(\hat{d}_t, d_t) + \lambda_f \,\mathrm{BCE}(\hat{f}_t, f_t). \tag{8}$$

where d_t and f_t are binary completion and failure labels, and $\hat{d}_t$ and $\hat{f}_t$ are the corresponding training predictions. The auxiliary failure loss improves the features used by the completion head. For value prediction, candidate action chunks A_t are encoded as tokens and processed jointly with the context x_t by the shared VLM. A learned query token aggregates these features for a 101-bin categorical value head, yielding

$$Q_\theta(x_t, A_t) = \sum_{b=1}^{101} z_b \, p_\theta(b \mid x_t, A_t), \tag{9}$$

where p_θ is the softmax output and z_b are fixed bin centers. The value expert uses a LoRA adapter [39] initialized from the state expert; only this adapter, the action encoder, query embedding, and value head are trained, while the shared backbone, existing experts, and WAM remain frozen.

Mixed-Frequency Coordination The three experts and the WAM operate at different frequencies, as illustrated in Fig. 4. The planner is invoked at episode initialization and near subtask boundaries to produce or update the remaining plan. After the WAM generates candidate action chunks, the value expert synchronously scores and fuses them into the action chunk executed by the robot. In contrast, the state expert runs asynchronously during action execution. Query-based status prediction requires only a single forward pass and reaches approximately 15 Hz. This allows state assessment to run concurrently with robot execution without blocking action execution. The resulting pipelined parallelism between action execution and state assessment improves inference throughput by 15%. If the completion probability reaches 0.5, the returned result triggers a subtask switch; stale results are discarded.

3.3 Joint Video–Action Modeling with MoTE

The WAM must jointly model the evolution of visual observations and the robot actions that cause these changes. In the original Mixture-of-Transformers (MoT), video and action branches exchange information through mixed attention: their queries, keys, and values are concatenated for joint attention, after which the outputs are returned to the two modality-specific branches [10], [11]. This mechanism enables cross-modal information flow, but leaves the post-attention transformations modality-specific and therefore provides limited capacity for learning shared video–action dynamics.

We introduce Mixture-of-Transformers Experts (MoTE) to strengthen this interaction, as illustrated in Fig. 5. MoTE applies a residual mixture-of-experts layer to the mixed-attention representation. The router produces sparse expert weights for the current modality:

$$\mathbf{w} = \mathrm{TopKSoftmax}(W_r \,\mathrm{LN}(\mathbf{H})), \tag{10}$$

where $\mathbf{H}$ is the mixed-attention representation and W_r is a modality-specific router. The routed residual update is

$$\widetilde{\mathbf{H}} = \mathbf{H} + \alpha \sum_{e \in \mathcal{E}} w_e E_e(\mathrm{LN}(\mathbf{H})), \tag{11}$$

where $\mathcal{E}$ denotes the selected experts, w_e is the routing weight, and E_e is an expert network. The modality-specific experts preserve the distinct statistics of video and action tokens, while the shared experts capture their common temporal and physical dynamics. Because the MoTE layer

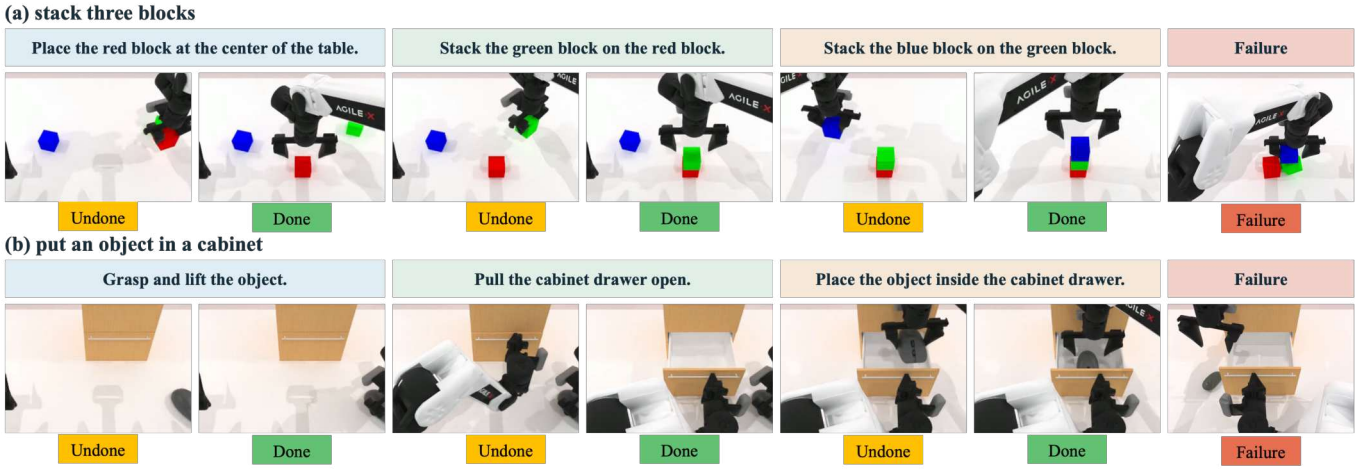


Fig. 3. Visualization of semantic subtask annotations, temporal boundaries, and failure cases in RoboTwin.

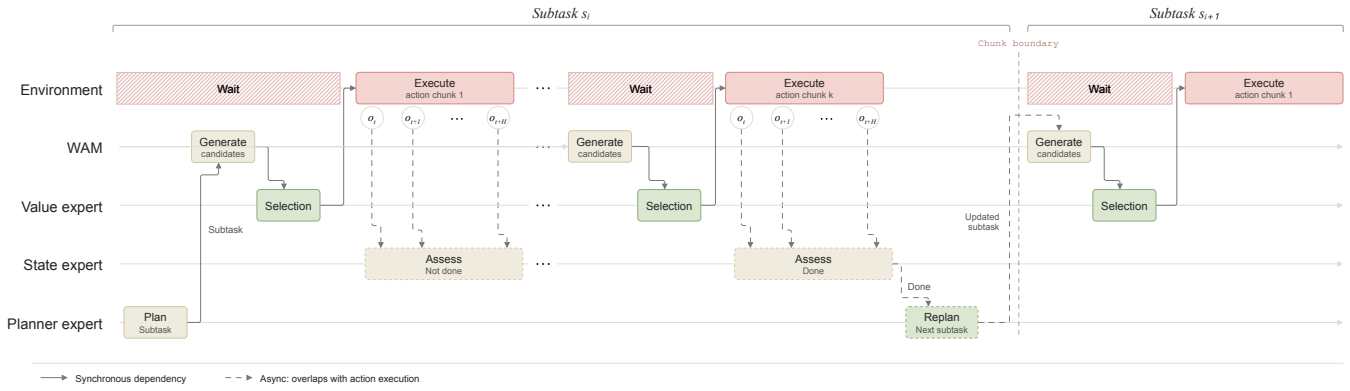


Fig. 4. Mixed-frequency coordination for progress-aware control in LAW. The planner operates at semantic subtask boundaries, while WAM generation and value-based selection synchronously determine each action chunk. The state expert asynchronously evaluates semantic subtask progress during action execution and triggers the next planning decision when the active subtask is completed.

is applied immediately after mixed attention, cross-modal features can be further transformed by both private and shared experts before entering the next Transformer block.

In the default configuration, each modality has two private experts and both modalities share one additional expert. The router uses top-1 selection and the residual scale is set to 0.1. The training objective combines video prediction, action prediction, and router regularization:

$$\mathcal{L}_{\text{WAM}} = \lambda_v \mathcal{L}_{\text{video}} + \lambda_a \mathcal{L}_{\text{action}} + \lambda_{\text{bal}} \mathcal{L}_{\text{balance}} + \lambda_z \mathcal{L}_z. \quad (12)$$

Here, $\mathcal{L}_{\text{balance}}$ encourages balanced expert usage and $\mathcal{L}_z$ regularizes the router logits. We set $\lambda_v = \lambda_a = 1$ and $\lambda_{\text{bal}} = \lambda_z = 10^{-4}$ in the default WAM configuration. MoTE therefore supplements attention-based video–action exchange with modality-specific and shared nonlinear transformations, improving the joint modeling of scene evolution and robot actions.

3.4 Dual-Stream Hierarchical Memory

Long-horizon manipulation requires historical context to assess task progress and maintain consistent action generation [4], [12]. Retaining every past observation is computationally expensive, while compressing the entire history into a single representation can discard recent motion details and critical subtask transitions. LAW therefore maintains separate VLM and WAM memory streams over the same selected observations.

At time t, we retain the previous five observations at a stride of four, giving the recent window $\{o_{t-20}, o_{t-16}, o_{t-12}, o_{t-8}, o_{t-4}\}$; unavailable elements are omitted near the beginning of an episode, and the current observation o_t is provided separately. We also retain the observation at the start of every encountered subtask when it falls outside this recent window. Earlier frames are sampled every four steps within the active subtask and every 16 steps within completed subtasks. All selected timestamps share the phase $t \bmod 4$, and a subtask-boundary frame is stored only once. Let $\mathcal{C}_t = (c_1, \ldots, c_m)$ denote the resulting long-term timestamps and $\mathcal{B}_t = (\beta_1, \ldots, \beta_p)$ the retained boundary timestamps, both in temporal order. Figure 6 illustrates how the WAM integrates short-term, subtask-boundary, and long-term visual memory with the current observation and action sequence.

The two streams encode these observations in their native token spaces. Let $\mathbf{v}_\tau = V(o_\tau)$ be the VLM visual tokens and $\mathbf{w}_\tau = W(o_\tau)$ the WAM VAE tokens. For an older observation, we compress its spatial tokens into a single

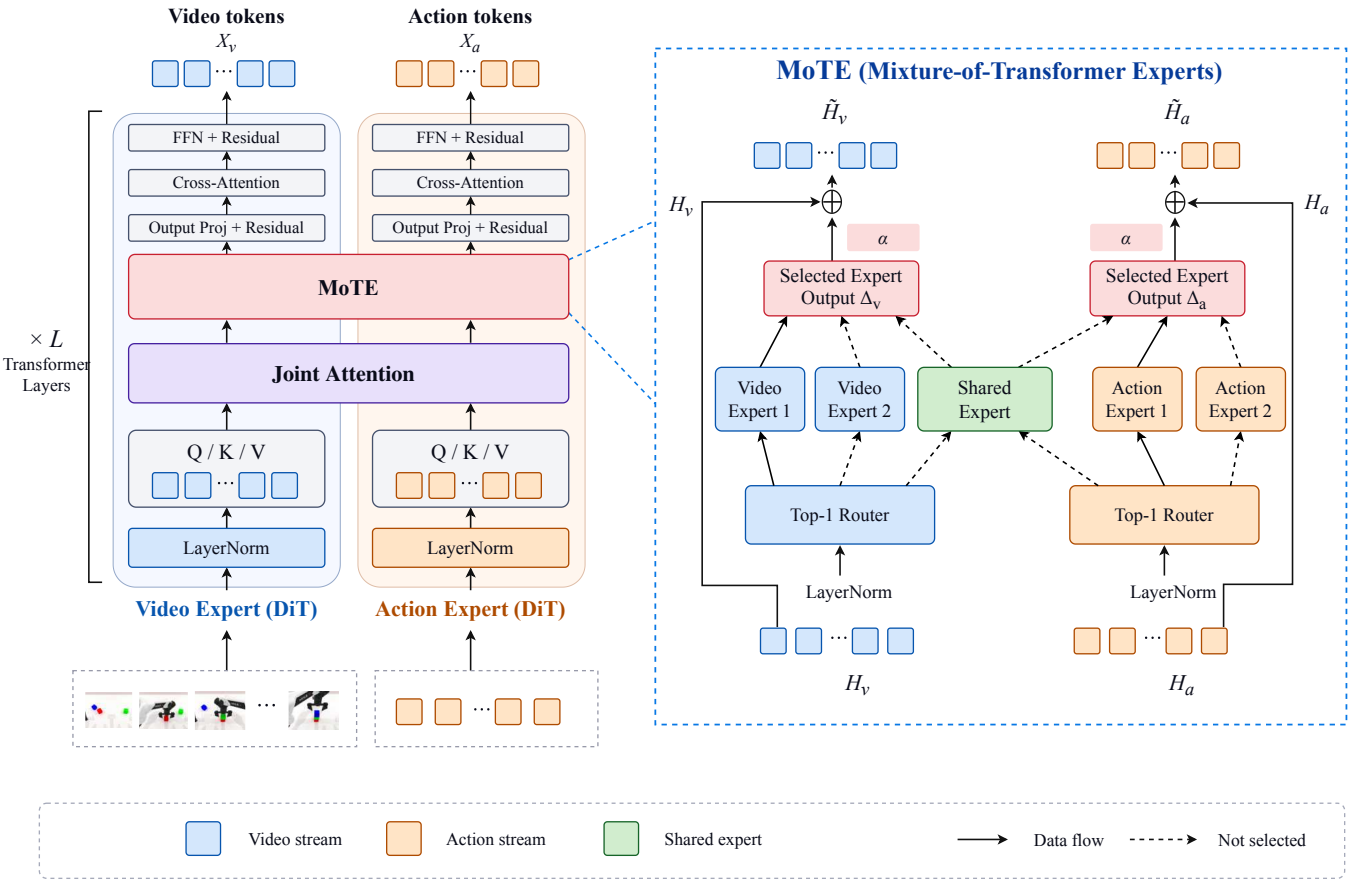


Fig. 5. MoTE architecture for joint video–action modeling. The video and action DiT branches exchange information through MoT mixed self-attention. After mixed attention, modality-specific top-1 routers select private experts or a shared expert available to both streams. The routed expert output is scaled by α and added residually before the remaining Transformer transformations.

summary:

$$\bar{\mathbf{v}}_\tau = \mathrm{LN}\left(\frac{1}{N_\tau}\sum_{j=1}^{N_\tau}\mathbf{v}_{\tau,j}\right), \qquad \bar{\mathbf{w}}_\tau = P_{\mathrm{W}}(\mathbf{w}_\tau), \tag{13}$$

where P_{W} maps an older VAE feature map to one token compatible with the WAM patch embedding. Boundary and recent observations retain their full spatial token sequences.

Let $\mathbf{x}_\tau^X$ and $\bar{\mathbf{x}}_\tau^X$ denote the full and compressed tokens of stream X, respectively, where $(\mathbf{x}^{\mathrm{V}}, \bar{\mathbf{x}}^{\mathrm{V}}) = (\mathbf{v}, \bar{\mathbf{v}})$ and $(\mathbf{x}^{\mathrm{W}}, \bar{\mathbf{x}}^{\mathrm{W}}) = (\mathbf{w}, \bar{\mathbf{w}})$. Using the same timestamps, both memory streams can then be written as

$$\mathcal{M}_t^X = \left[\{\bar{\mathbf{x}}_{c_i}^X\}_{i=1}^m; \{\mathbf{x}_{\beta_i}^X\}_{i=1}^p; \{\mathbf{x}_{t-4j}^X\}_{j=1}^5\right], X \in \{\mathrm{V}, \mathrm{W}\}. \tag{14}$$

The three groups correspond to compressed long-term history, full-resolution subtask boundaries, and full-resolution recent observations. Thus, $\mathcal{M}_t^{\mathrm{V}}$ supports planning, state estimation, and candidate evaluation, while $\mathcal{M}_t^{\mathrm{W}}$ conditions video prediction and action generation.

3.5 Subtask-Local Value Learning

Subtask-local value supervision. Episode-level outcomes discard useful supervision when a trajectory completes several subtasks before failing. We instead assign reward 1 to the terminal action of each successfully completed subtask and 0 elsewhere. Completed prefixes therefore retain positive supervision, while the final failed subtask receives no positive reward and unobserved subtasks are excluded. For a chunk with h_t valid actions, the TD target is

$$y_t = \sum_{i=0}^{h_t-1}\gamma^i r_{t+i} + b_t\gamma^{h_t}Q_{\bar{\theta}}(x_{t+h_t}, A_{t+h_t}^{\mathrm{data}}), \tag{15}$$

where $A_{t+h_t}^{\mathrm{data}}$ is the next recorded chunk and $\bar{\theta}$ denotes exponential-moving-average target parameters. Chunks are truncated and masked at subtask boundaries, with $b_t = 0$ whenever the terminal action is included. Thus, bootstrapping remains within the current subtask.

Building on categorical value learning [40], we clip y_t to the value support, obtaining $\tilde{y}_t$, and construct soft labels using a normalized Gaussian kernel evaluated at the bin centers. The value expert is trained with categorical cross-entropy:

$$\begin{aligned}\rho_{t,b} &\propto \exp\left[-\frac{(z_b - \tilde{y}_t)^2}{2\sigma^2}\right], \\ \mathcal{L}_{\mathrm{value}} &= -\mathbb{E}_{\mathcal{D}}\left[\sum_{b=1}^{101}\rho_{t,b}\log p_\theta(b \mid x_t, A_t)\right],\end{aligned} \tag{16}$$

where ρ_t is normalized across bins and treated as a stop-gradient target.

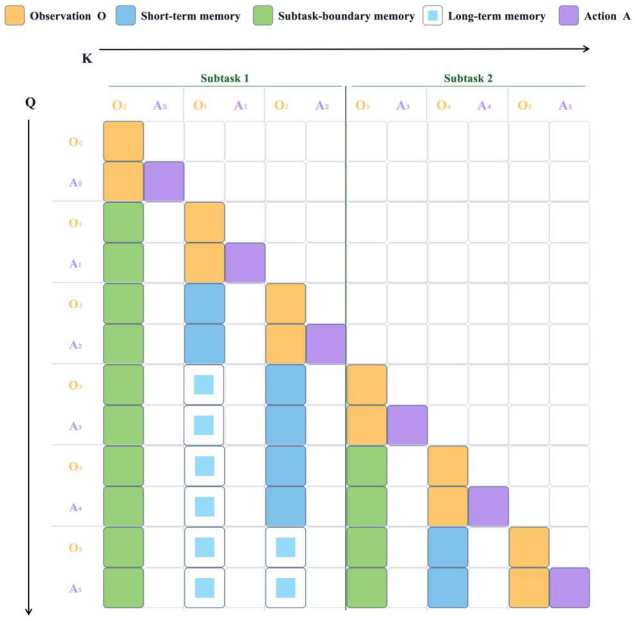


Fig. 6. Hierarchical visual memory and attention pattern in the WAM. Short-term and subtask-boundary observations are retained at full resolution, while older observations are spatially compressed.

Value-guided action selection. At execution time, the WAM generates candidate chunks $\{A_t^{(n)}\}_{n=1}^{N}$. The value expert retains the top-K index set $\mathcal{E}_t$ and assigns temperature-controlled weights

$$w_n = \frac{\exp\Big(Q_\theta(x_t, A_t^{(n)})/T\Big)}{\sum_{k\in\mathcal{E}_t}\exp\Big(Q_\theta(x_t, A_t^{(k)})/T\Big)}, \qquad n \in \mathcal{E}_t, \tag{17}$$

where $T > 0$ is the temperature. Each weight is shared across the candidate's time steps. Positions and gripper commands are averaged linearly. For each arm and time step, normalized candidate quaternions q_n are combined using the weighted Markley mean [41]:

$$M = \sum_{n\in\mathcal{E}_t} w_n q_n q_n^\top, \qquad \widehat{q} \in \arg\max_{\|q\|_2=1} q^\top M q. \tag{18}$$

The aggregate is a unit eigenvector of M associated with its largest eigenvalue, invariant as a rotation to candidate sign flips $q_n \mapsto -q_n$. The robot executes a prefix of the aggregated chunk before repeating the process with new observations.

4 Experiments

4.1 Training Details

Training datasets. We use task-specific training data from RoboTwin [13], RoboDojo [14], and real-world manipulation. RoboTwin covers 50 bimanual manipulation tasks; its public Aloha-AgileX release provides 50 clean and 500 domain-randomized demonstrations per task. The official RoboDojo training release contains 3,500 trajectories across 35 task directories, with 100 trajectories per directory collected through automated synthesis or VR-based teleoperation. We augment the simulation trajectories with semantic subtask annotations and temporal boundaries for planning and completion assessment.

For real-world training, we collect 300 teleoperated trajectories per task on the AgileX Cobot Magic platform, totaling 900 trajectories, with one scene camera and two wrist cameras. Puzzle comprises three subtasks, each inserting one colored piece into its matching recess. Domino comprises five subtasks: standing four dominoes individually and then toppling the row. Organize comprises nine subtasks: opening the corresponding drawer, placing a colored block inside, and closing the drawer, repeated for three blocks. Each trajectory includes multiview observations, robot states, actions, language instructions, and semantic subtask boundaries. Successful and failed trajectories provide complementary supervision for value learning. Additional annotation and collection details are given in Appendices C and B.2.

Overall training pipeline. LAW is trained in two stages. In the first stage, we train the planner and state experts sequentially on semantic subtask annotations, while training the WAM independently on paired video and robot-action sequences. In the second stage, we freeze these components and train the value expert on successful demonstrations and annotated failure trajectories using the subtask-local objective in Section 3.5. All parameters are frozen for evaluation. This two-stage procedure prevents value learning from destabilizing the action generator or the planner and state experts.

Hybrid-Expert VLM. We use Qwen3-VL-2B-Instruct [42] as the shared VLM backbone and keep its language and vision parameters frozen. The planner and state adapters use rank $r = 64$, while the value adapter uses rank $r = 128$, adding approximately 100M parameters in total. The planner is trained first for 3,000 optimizer steps using a rank-64 LoRA adapter with scale 128 and dropout 0.05. We then freeze the planner adapter and train the state adapter and its completion and auxiliary failure heads for 50,000 steps using the same LoRA configuration. Both stages use AdamW with a learning rate of 10^{-5}, bfloat16 precision, gradient checkpointing, and a maximum sequence length of 8,192. Training examples contain the main, left-wrist, and right-wrist views together with the hierarchical memory described in Section 3.4. The VLM stages are trained with DeepSpeed ZeRO-1 on 16 GPUs.

After restoring the trained planner and state adapters, we freeze the shared backbone, both existing experts, and the WAM. The value adapter is initialized from the state adapter, while the action encoder, value query, and categorical head are newly initialized. Each input contains a 32-step, 16-dimensional end-effector action chunk sampled at 0.06 seconds per step. Training mixes successful demonstrations with annotated failure trajectories, with failures comprising 25% of the samples. We train for 50,000 steps using AdamW, a learning rate of 10^{-5}, zero weight decay, 1,000 warm-up steps followed by linear decay, and gradient clipping at 1.0. The categorical critic uses 101 bins over $[-0.01, 1.01]$, discount factor $\gamma = 0.99$, HL-Gauss width $\sigma = 0.0075$, and an exponential-moving-average target with update rate $\tau = 0.005$.

Failure data construction. We construct failure trajectories in simulation by replaying successful demonstrations and

TABLE 1
Training hyperparameters of LAW.

Stage	Steps	LR	Batch	Acc.	GPUs
Planner expert	3,000	1×10^{-5}	6	1	16
State expert	50,000	1×10^{-5}	6	1	16
WAM	30,000	1×10^{-4}	6	2	32
Value expert	50,000	1×10^{-5}	8	1	16

injecting task-specific perturbations at selected manipulation stages. Each task is assigned one to three applicable failure modes, including missed grasps, premature releases, incorrect placements, missed contacts, and incomplete or misaligned task-specific operations. For each task–failure-type pair, we target 35 clean trajectories and 100 domain-randomized trajectories. We then filter out semantically invalid, visually ambiguous, and low-quality samples.

World-action model. The WAM is initialized from Wan2.2-TI2V-5B, with the action branch initialized from the pretrained ActionDiT weights. We optimize the video and action DiTs, including the MoTE layers, while keeping the remaining pretrained components frozen. Training uses three camera views at 384×384 resolution, an action horizon of 32, and subtask-conditioned text embeddings. We train for 30,000 optimizer steps on 32 GPUs with a per-GPU batch size of 6 and two-step gradient accumulation. We use AdamW with $\beta_1 = 0.9$, $\beta_2 = 0.95$, a learning rate of 10^{-4}, weight decay 10^{-2}, and gradient clipping at 1.0. The learning rate is linearly warmed up for the first 5% of training and then follows a cosine schedule. Training uses bfloat16 precision and DeepSpeed ZeRO-1.

Testing. At test time, all modules are frozen. The WAM samples 32 candidate chunks using three denoising steps, without explicitly generating future images. The value expert retains the top eight candidates with temperature $T = 1$, and the robot executes a prefix of their weighted aggregate before generating new candidates from the next observation.

Table 1 summarizes the main optimization settings. Batch size denotes the number of samples per GPU, and "Acc." denotes gradient accumulation steps.

4.2 Main Results

We evaluate LAW on 50 RoboTwin tasks under clean and randomized settings, comparing it with representative mainstream robot policies. As shown in Table 2, LAW achieves success rates of 94.22% and 94.20%, respectively, yielding an average of 94.21% and outperforming the selected baselines in both settings. Without robot-data pretraining, LAW improves over Fast-WAM (91.8%) by 2.4 percentage points and exceeds LingBot-VA 2.0 (93.6%), the highest-performing pretrained method among the selected baselines.

Table 3 reports LAW's performance on RoboDojo across Generalization, Precision, Long-Horizon, Memory, and Open. LAW achieves the highest score and success rate in all five dimensions among the listed methods without additional robot-domain or embodied-interaction pretraining, averaging 6.47 / 3.82%. On Long-Horizon, LAW reaches 11.16 / 6.67%, exceeding pretrained LDA-1B, RDT-1B, and H-RDT. On Memory, it achieves 7.46 / 5.44%, surpassing all listed pretrained methods, including X-WAM (6.32 / 4.67%). These results support the effectiveness of semantic subtask coordination and hierarchical memory for sustained execution and history-dependent decisions.

TABLE 2
Comparison with mainstream robot policies on RoboTwin.

Method	Embodied PT.	Clean (%)	Rand. (%)	Avg. (%)
X-VLA [43]	√	72.9	72.8	72.9
$\pi_{0.5}$ [44]	√	82.7	76.8	79.8
Qwen-RobotManip [45]	√	93.4	92.5	93.0
Motus [46]	√	88.7	87.0	87.9
Fast-WAM [11]	×	91.9	91.8	91.8
LingBot-VA [10]	√	92.9	91.5	92.2
LingBot-VA 2.0 [33]	√	93.8	93.4	93.6
LAW + Fast-WAM	×	**94.22**	**94.20**	**94.21**

Embodied PT.: robot-data pretraining.

We further evaluate LAW on three real-world manipulation tasks with distinct sequential requirements: *Puzzle*, which involves inserting three pieces into a board; *Domino*, which requires standing four dominoes in a row and then toppling them; and *Organize*, which requires placing three colored blocks into their matching drawers and closing each drawer. These tasks comprise three, five, and nine semantic subtasks, respectively. We compare LAW with the VLA baseline $\pi_{0.5}$ and the WAM baseline Fast-WAM using complete-task success rate and a task score that assigns equal credit to completed subtasks and normalizes progress to 0–100. As shown in Fig. 7, LAW outperforms both baselines on every task under both metrics. Averaged across the three tasks, LAW achieves a success rate of 65% and a task score of 84.06, compared with 28.33% and 55.50 for $\pi_{0.5}$, and 25% and 51.87 for Fast-WAM. On Organize, the task with the longest subtask sequence, LAW achieves 65% success and a score of 85.54, exceeding the higher-performing of the two baselines, Fast-WAM, by 25 percentage points and 15.57 points, respectively. These results demonstrate improved complete-task execution and subtask completion across real-world manipulation sequences of varying length.

4.3 Component Ablations

We progressively add the hybrid-expert architecture, supervision with failure trajectories as negative samples, and hierarchical memory to isolate their effects on VLM planning and state estimation. Planning is measured by normalized edit distance (NED), BLEU, and normalized exact match (EM) of the predicted subtask sequence. For state estimation, we report completion accuracy, recall, and class-frequency-weighted F1 score.

Table 4 shows that separating control functions with hybrid experts substantially improves both planning and state estimation. Adding failure trajectories as negative samples further reduces completion errors, while hierarchical memory achieves the strongest overall performance. Figure 8 provides a more detailed view of this progression. The false-positive rate decreases from 11.4% to 0.7%, and the false-negative rate decreases from 32.8% to 2.8%. These results

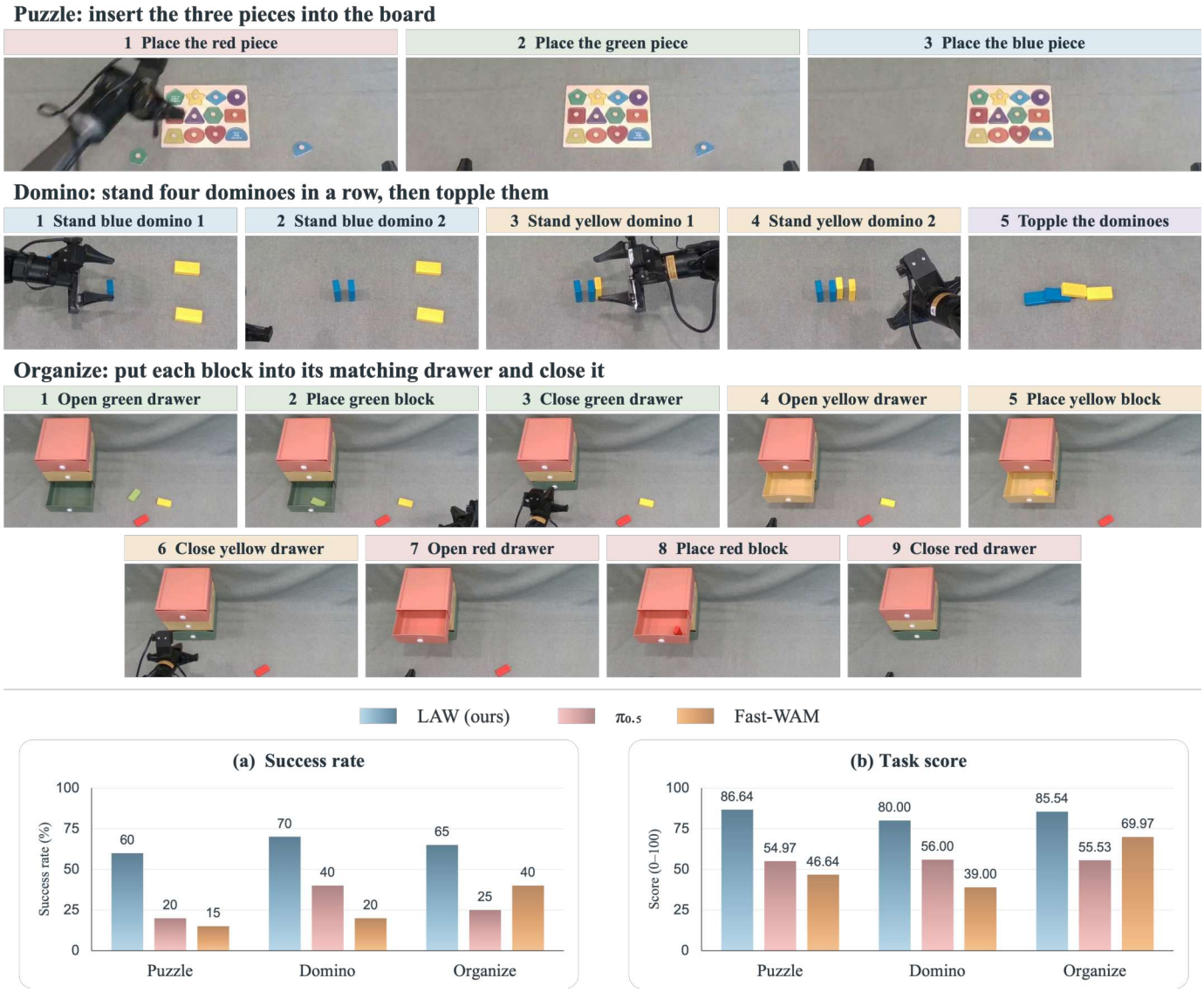


Fig. 7. Real-world manipulation results. Representative executions illustrate the semantic subtasks of Puzzle (three), Domino (five), and Organize (nine). (a) Complete-task success rate. (b) Task score, computed from the fraction of completed subtasks and scaled to 0–100. LAW is compared with $\pi_{0.5}$ and Fast-WAM.

indicate that the complete model is both less likely to switch subtasks prematurely and less likely to miss a completed subtask.

We further ablate the components that improve end-to-end task success. Table 5 includes two independent "only" variants and a cumulative sequence. The MoTE-only and subtask-only rows add a single component to Fast-WAM, whereas each subsequent row retains all components enabled above it.

MoTE and subtask conditioning provide comparable gains when introduced independently. Combining them raises the average success rate to 93.16%, hierarchical memory further improves it to 93.80%, and value-guided action selection achieves the best result of 94.21%. These cumulative gains show that joint video–action modeling, semantic task structure, temporal context, and value-based candidate selection contribute complementary improvements.

5 Analysis

5.1 Effect of VLM Choice

We examine how the choice of VLM affects the LAW agent while keeping the WAM backbone and overall control framework fixed. Table 6 compares the default 2B hybrid-expert VLM with variants using Embodied-R1.5 and GPT-5.5 High. The default VLM achieves 94.21% average success on RoboTwin, compared with 88.8% and 88.7%, respectively, corresponding to gains of 5.41 and 5.51 percentage points. Its performance is also consistently higher in both clean and randomized settings.

These results suggest that embodied reasoning capability alone is insufficient for reliable closed-loop manipulation. The VLM must coordinate remaining-subtask planning, completion assessment, and candidate-action evaluation using the current objective and execution history. The advantage of the default VLM is consistent with its specialized experts and task-aligned supervision for these functions.

TABLE 3
Comparison across RoboDojo capability dimensions.

Method	Embodied PT.	Score / Success (%) Generalization	Precision	Long-Horizon	Memory	Open	Average
GigaWorld-Policy [47]	Yes	5.34 / 2.89	6.15 / 1.83	15.51 / 8.92	3.46 / 2.22	0.54 / 0.50	6.20 / 3.27
LDA-1B [48]	Yes	0.71 / 0.17	3.21 / 0.50	1.92 / 0.08	2.08 / 1.78	0.00 / 0.00	1.58 / 0.51
RDT-1B [49]	Yes	0.56 / 0.33	0.38 / 0.00	1.13 / 0.00	0.49 / 0.33	0.00 / 0.00	0.51 / 0.13
H-RDT [50]	Yes	0.49 / 0.22	0.41 / 0.00	2.23 / 0.17	0.12 / 0.11	0.08 / 0.08	0.67 / 0.12
X-WAM [51]	Yes	**7.39** / 3.33	**6.72** / 1.83	**17.47** / 9.08	**6.32** / 4.67	**0.57** / 0.25	**7.69** / 3.83
HDR-WAM [52]	No	4.98 / 3.17	5.90 / 2.25	9.85 / 4.75	6.65 / 4.67	0.50 / 0.50	5.47 / 3.00
AHA-WAM [53]	No	5.79 / 3.28	5.86 / 2.42	8.61 / 2.67	2.97 / 2.78	0.88 / 0.83	4.82 / 2.39
Fast-WAM [11]	No	2.34 / 1.11	1.96 / 0.00	9.14 / 5.17	3.55 / 3.44	0.42 / 0.42	3.48 / 2.03
ACT [54]	No	0.69 / 0.56	0.85 / 0.00	1.73 / 0.92	1.65 / 0.13	0.00 / 0.00	0.98 / 0.32
LAW (Ours)	**No**	**5.93 / 3.39**	**6.27 / 2.58**	**11.16 / 6.67**	**7.46 / 5.44**	**1.53 / 1.00**	**6.47 / 3.82**

Results are averaged over three seeds. Embodied PT.: additional robot-domain or embodied-interaction pretraining. Bold: best score within each group. HDR-WAM adapts HDR to WAM.

TABLE 4
Ablation of VLM experts, failure data, and hierarchical memory.

Settings			Planning (%)			State Estimation (%)		
Hybrid Expert	Failure Data	Memory	NED↓	BLEU↑	EM↑	Accuracy↑	Recall↑	Weighted F1↑
×	×	×	1.35	98.12	96.36	87.16	67.16	89.34
√	×	×	0.34	99.58	99.27	98.10	95.48	98.12
√	√	×	0.34	99.58	99.27	98.48	95.63	98.49
√	√	√	**0.12**	**99.77**	**99.64**	**99.20**	**97.22**	**99.22**

TABLE 5
Ablation of LAW components on RoboTwin.

MoTE	Subtask	Memory	Value Guidance	Clean (%)	Rand. (%)	Avg. (%)
×	×	×	×	91.88	91.78	91.80
√	×	×	×	93.02	92.31	92.67
×	√	×	×	92.84	92.52	92.68
√	√	×	×	93.66	92.66	93.16
√	√	√	×	93.78	93.82	93.80
√	√	√	√	**94.22**	**94.20**	**94.21**

"Only": a single component added to Fast-WAM; other rows add components cumulatively.

TABLE 6
Effect of VLM choice on RoboTwin.

VLM	Clean (%)	Rand. (%)	Avg. (%)
Embodied R1.5	88.6	89.0	88.8
GPT-5.5 High	89.7	87.7	88.7
LAW VLM (Ours)	**94.22**	**94.20**	**94.21**

5.2 MoE Placement and Shared Experts

We investigate whether the improvement of MoTE comes merely from increasing model capacity or from where and how the experts process video–action features. Table 7 compares MoE layers inserted at conventional attention and FFN locations with MoTE applied after MoT mixed attention. Attn-MoE and FFN-MoE add expert capacity but remain close to the no-MoE baseline, indicating that additional parameters alone do not yield a meaningful improvement.

The placement of MoTE is critical because it transforms the representation immediately after video and action tokens exchange information through mixed attention. Experts placed in generic attention or FFN positions mainly increase per-branch capacity, whereas MoTE operates on representations that already contain cross-modal information. Figure 9 illustrates the observed loss trends: after the common early descent, full MoTE maintains the lowest

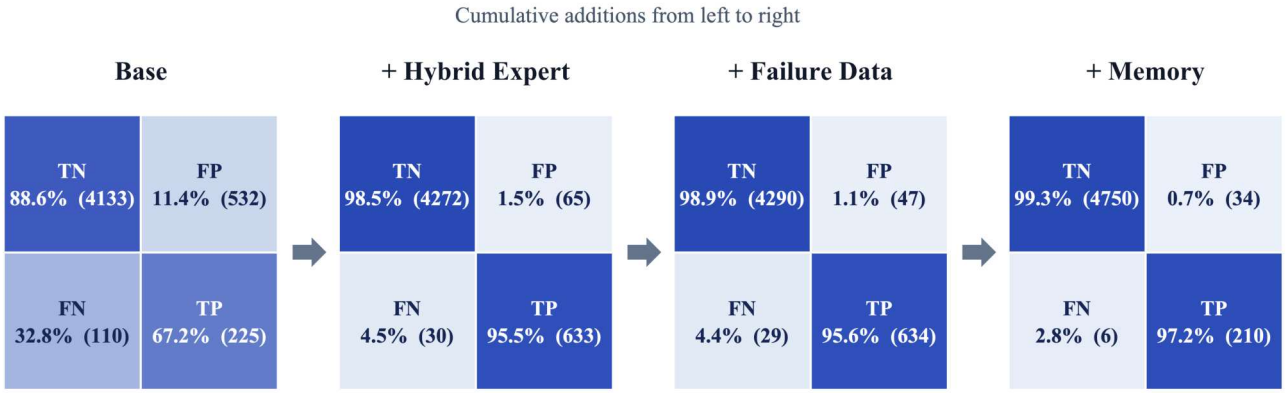


Fig. 8. Cumulative ablation of subtask-completion prediction from left to right. Each row-normalized matrix adds one component to the preceding setting; cells report true negatives (TN), false positives (FP), false negatives (FN), and true positives (TP), with sample counts in parentheses.

TABLE 7
Effect of MoE placement and shared experts on RoboTwin.

Setting	Clean (%)	Rand. (%)	Avg. (%)
W/o MoE	91.88	91.78	91.80
Attn-MoE	91.77	91.92	91.85
FFN-MoE	92.03	91.85	91.94
MoTE w/o shared	91.98	92.07	91.93
MoTE	**93.02**	**92.31**	**92.67**

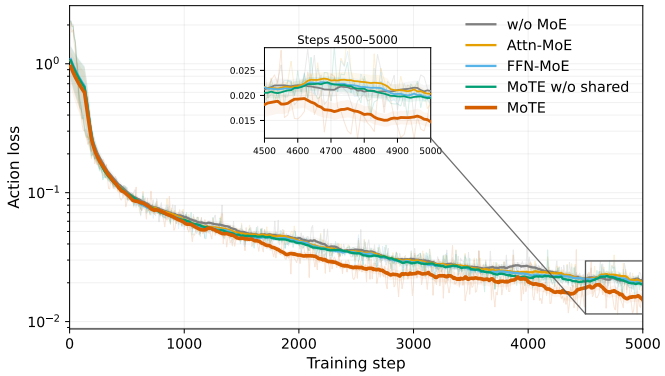


Fig. 9. Action-loss curves for different expert configurations.

action loss throughout the later stage. Attn-MoE and FFN-MoE remain close to the no-MoE baseline, while removing the shared expert also yields a higher action loss than full MoTE. This behavior is consistent with the success-rate comparison in Table 7.

The comparison with MoTE without a shared expert further supports this interpretation. Adding the shared expert improves the average success rate from 91.93% to 92.67%. The modality-specific experts preserve the distinct statistics of video and action tokens, while the shared expert captures transformations common to both modalities. Together, the lower loss and higher success rate attribute the gain of MoTE to joint video–action modeling rather than model capacity alone.

5.3 Subtask Granularity

We compare task-level control, low-level subtasks, and semantic subtasks to examine how subtask granularity affects long-horizon manipulation. The model without subtasks directly conditions control on the overall instruction. Low-level subtasks introduce finer stage partitions, whereas semantic-level subtasks align each stage with a meaningful manipulation objective that can be shared by the planner, state expert, and memory system. Table 8 reports a ten-task diagnostic comparison.

Semantic-level subtasks achieve the highest ten-task average, improving from 92.4% without subtasks and 89.1% with low-level subtasks to 95.8%. The largest gains appear on tasks such as Handover Block, Place Can Basket, and Place Object Basket, where progress depends on recognizing distinct manipulation stages. Finer partitions can fragment a coherent semantic objective into short transitions, making completion decisions and historical context less stable. Semantic stages instead provide consistent units for planning, asynchronous state assessment, and hierarchical memory. The semantic decomposition does not improve every saturated task, but its aggregate advantage indicates that the abstraction level of the subtask, rather than decomposition alone, is important for long-horizon control.

5.4 Subtask-Local Value Learning Ablation

We compare action selection without value guidance, value experts trained with episode-level supervision, and value experts trained with subtask-local supervision on the same ten RoboTwin tasks. Table 9 reports the resulting success rates under clean and randomized settings.

Subtask-local supervision achieves the highest average success rate of 97.90%, compared with 96.30% without value guidance and 96.55% with episode-level supervision, improving by 1.60 and 1.35 percentage points, respectively. The gains hold in both clean and randomized settings. Compared with episode-level supervision, subtask-local learning improves Place A2B Left from 95.5% to 97.5%, Place Bread Basket from 95.0% to 97.0%, and Open Microwave from 94.0% to 95.5%. These results support assigning credit at semantic subtask boundaries: completed stages of failed

TABLE 8
Subtask-granularity ablation on ten RoboTwin tasks.

Task	W/o Subtask			Low-Level Subtasks			Semantic-Level Subtasks		
	Clean (%)	Rand. (%)	Avg. (%)	Clean (%)	Rand. (%)	Avg. (%)	Clean (%)	Rand. (%)	Avg. (%)
Place Can Basket	71	69	70.0	54	60	57.0	**85**	**88**	**86.5**
Handover Block	95	81	88.0	69	82	75.5	**100**	**96**	**98.0**
Rotate QRcode	**93**	89	91.0	87	91	89.0	**93**	**97**	**95.0**
Open Laptop	98	**100**	99.0	99	95	97.0	**100**	99	**99.5**
Stack Blocks Two	**100**	**100**	**100.0**	91	98	94.5	**100**	**100**	**100.0**
Place Burger Fries	96	**99**	**97.5**	**99**	96	**97.5**	97	98	**97.5**
Put Bottles Dustbin	**95**	90	92.5	92	**94**	**93.0**	91	**94**	92.5
Place Container Plate	96	**100**	98.0	**100**	97	**98.5**	98	98	98.0
Place Object Basket	89	88	88.5	89	90	89.5	**93**	**92**	**92.5**
Move Pillbottle Pad	**100**	**99**	**99.5**	**100**	**99**	**99.5**	98	**99**	98.5
10-task Average	93.3	91.5	92.4	88.0	90.2	89.1	**95.5**	**96.1**	**95.8**

TABLE 9
Value-supervision ablation on ten RoboTwin tasks.

Task	W/o Value Guidance			Episode-Level Supervision			Subtask-Level Supervision		
	Clean (%)	Rand. (%)	Avg. (%)	Clean (%)	Rand. (%)	Avg. (%)	Clean (%)	Rand. (%)	Avg. (%)
Move PillBottle Pad	**100**	99	99.5	99	**100**	99.5	**100**	**100**	**100.0**
Place Cans Plasticbox	**100**	97	98.5	**100**	99	99.5	**100**	**100**	**100.0**
Stamp Seal	99	**99**	99.0	99	98	98.5	**100**	**99**	**99.5**
Place Burger Fries	98	97	97.5	97	**99**	98.0	**99**	**99**	**99.0**
Place A2B Left	94	95	94.5	96	95	95.5	**99**	**96**	**97.5**
Place Phone Stand	96	95	95.5	97	94	95.5	**99**	**96**	**97.5**
Beat Block Hammer	**97**	95	96.0	**97**	93	95.0	96	**99**	**97.5**
Place Bread Basket	96	**96**	96.0	94	**96**	95.0	**99**	95	**97.0**
Open Microwave	94	**96**	95.0	**97**	91	94.0	96	95	**95.5**
Place Mouse Pad	92	91	91.5	**95**	95	95.0	**95**	**96**	**95.5**
10-task Average	96.60	96.00	96.30	97.10	96.00	96.55	**98.30**	**97.50**	**97.90**

trajectories retain positive supervision, allowing the value expert to evaluate candidate actions against the active objective rather than relying solely on the final episode outcome.

5.5 Memory Component Ablation

We cumulatively ablate the three components of hierarchical memory on the RoboDojo Memory dimension. Short-term memory retains the recent observation window, long-term memory adds compressed older observations, and subtask-boundary memory further preserves full-resolution observations at semantic transitions.

Table 10 shows consistent gains from adding each memory component. Short-term memory raises the score from 4.06 to 5.57 and the success rate from 3.44% to 4.00%, indicating the value of recent observations for history-dependent decisions. Adding compressed long-term memory further improves these metrics to 6.48 and 4.89%, suggesting that older context complements the recent observation window. Preserving full-resolution subtask-boundary observations yields the highest score and success rate of 7.46 and 5.44%. These cumulative improvements support retaining both recent visual details and broader historical context, while highlighting the additional benefit of preserving observations at semantic transitions.

TABLE 10
Hierarchical-memory ablation on RoboDojo.

Setting	Score	Success (%)
W/o memory	4.06	3.44
+ Short-term memory	5.57	4.00
+ Long-term memory	6.48	4.89
+ Subtask-boundary memory	**7.46**	**5.44**

Results are averaged over six tasks and three seeds.

5.6 WAM Backbone Extensibility

LAW interacts with the action generator through a common WAM interface, allowing its planning, state estimation, value selection, and memory components to be combined with different WAM backbones. Table 11 evaluates this extensibility on RoboTwin using the default Fast-WAM backbone and OpenWAM-α [55]. LAW improves the average success rate of Fast-WAM from 91.85% to 94.21%, and that of

TABLE 11
Comparison of WAM backbones on RoboTwin.

Method	Clean (%)	Rand. (%)	Avg. (%)
Flex-π [56]	94.30	94.80	94.60
MotuBrain [57]†	95.80	96.08	95.94
Fast-WAM	91.90	91.80	91.85
LAW + Fast-WAM	**94.22**	**94.20**	**94.21**
OpenWAM-α	93.74	93.46	93.60
LAW + OpenWAM-α	**95.60**	**95.52**	**95.56**

†: closed-source methods

OpenWAM-α from 93.60% to 95.56%. LAW + OpenWAM-α establishes a new open-source state of the art on RoboTwin with 95.56% average success. These results demonstrate LAW's extensibility across WAM backbones: its progress-aware control mechanisms improve both action generators, while integration with a stronger pretrained WAM further raises the framework's achievable performance.

6 Conclusion

This paper presented LAW, a closed-loop VLM–WAM framework that unifies language reasoning, action generation, and world modeling through semantic subtask progress. The LAW agent coordinates specialized planning, completion assessment, and value-guided action fusion at complementary frequencies, connecting execution feedback to continued control and subtask transitions. MoTE strengthens joint video–action modeling through shared and modality-specific experts, while dual-stream hierarchical memory preserves recent context and semantic boundary observations. Subtask-local value learning further retains useful supervision from completed stages of failed trajectories. LAW achieves 94.21% average success on RoboTwin and 65% on three real-robot tasks, with strong performance on RoboDojo's memory and long-horizon capabilities. Integrating OpenWAM-α raises RoboTwin success to 95.56%, establishing an open-source state of the art and demonstrating that stronger pretrained WAMs can further improve LAW's performance. These results highlight semantic subtask progress as an effective interface for agentic physical intelligence. Future work will investigate transfer across robot embodiments and adaptive subtask decomposition for longer and more diverse tasks.

REFERENCES

[1] M. Ahn, A. Brohan, N. Brown, Y. Chebotar, O. Cortes, B. David, C. Finn, C. Fu, K. Gopalakrishnan, K. Hausman, A. Herzog, D. Ho, J. Hsu, J. Ibarz, B. Ichter, A. Irpan, E. Jang, R. Jauregui Ruano, K. Jeffrey, S. Jesmonth, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, K.-H. Lee, S. Levine, Y. Lu, L. Luu, C. Parada, P. Pastor, J. Quiambao, K. Rao, J. Rettinghouse, D. Reyes, P. Sermanet, N. Sievers, C. Tan, A. Toshev, V. Vanhoucke, F. Xia, T. Xiao, P. Xu, M. Yan, and A. Zeng, "Do as i can, not as i say: Grounding language in robotic affordances," in *Proceedings of The 6th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 205. PMLR, 2023, pp. 287–298. [Online]. Available: https://proceedings.mlr.press/v205/ichter23a.html

[2] W. Huang, F. Xia, T. Xiao, H. Chan, J. Liang, P. Florence, A. Zeng, J. Tompson, I. Mordatch, Y. Chebotar, P. Sermanet, N. Brown, T. Jackson, L. Luu, S. Levine, K. Hausman, and B. Ichter, "Inner monologue: Embodied reasoning through planning with language models," *arXiv preprint arXiv:2207.05608*, 2022. [Online]. Available: https://arxiv.org/abs/2207.05608

[3] Z. Liu, A. Bahety, and S. Song, "REFLECT: Summarizing robot experiences for failure explanation and correction," *arXiv preprint arXiv:2306.15724*, 2023.

[4] M. Torne, K. Pertsch, H. Walke, K. Vedder, S. Nair, B. Ichter, A. Z. Ren, H. Wang, J. Tang, K. Stachowicz *et al.*, "MEM: Multi-scale embodied memory for vision language action models," *arXiv preprint arXiv:2603.03596*, 2026.

[5] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia *et al.*, "RT-2: Vision-language-action models transfer web knowledge to robotic control," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 2165–2183. [Online]. Available: https://proceedings.mlr.press/v229/zitkovich23a.html

[6] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. P. Foster, P. R. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, "Openvla: An open-source vision-language-action model," in *Proceedings of The 8th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 270. PMLR, 2024, pp. 2679–2713. [Online]. Available: https://mlanthology.org/corl/2024/kim2024corl-openvla/

[7] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, H. Wang, and U. Zhilinsky, "π_0: A vision-language-action flow model for general robot control," *arXiv preprint arXiv:2410.24164*, 2024. [Online]. Available: https://arxiv.org/abs/2410.24164

[8] C. Zhu, R. Yu, S. Feng, B. Burchfiel, P. Shah, and A. Gupta, "Unified world models: Coupling video and action diffusion for pretraining on large robotic datasets," *arXiv preprint arXiv:2504.02792*, 2025. [Online]. Available: https://arxiv.org/abs/2504.02792

[9] R. Zheng, J. Wang, S. Reed, J. Bjorck, Y. Fang, F. Hu, J. Jang, K. Kundalia, Z. Lin, L. Magne, A. Narayan, Y. L. Tan, G. Wang, Q. Wang, J. Xiang, Y. Xu, S. Ye, J. Kautz, F. Huang, Y. Zhu, and L. Fan, "Flare: Robot learning with implicit world modeling," *arXiv preprint arXiv:2505.15659*, 2025. [Online]. Available: https://arxiv.org/abs/2505.15659

[10] Q. Zhang *et al.*, "Lingbot-va: Causal world modeling for robot control," *arXiv preprint arXiv:2601.21998*, 2026. [Online]. Available: https://arxiv.org/abs/2601.21998

[11] T. Yuan, Z. Dong, Y. Liu, and H. Zhao, "Fast-wam: Do world action models need test-time future imagination?" *arXiv preprint arXiv:2603.16666*, 2026. [Online]. Available: https://arxiv.org/abs/2603.16666

[12] H. Shi, B. Xie, Y. Liu, L. Sun, F. Liu, T. Wang, E. Zhou, H. Fan, X. Zhang, and G. Huang, "MemoryVLA: Perceptual-cognitive memory in vision-language-action models for robotic manipulation," in *International Conference on Learning Representations*, vol. 2026, 2026, pp. 18 567–18 602.

[13] T. Chen, Z. Chen, B. Chen, Z. Cai, Y. Liu, Z. Li, Q. Liang, X. Lin, Y. Ge, Z. Gu *et al.*, "RoboTwin 2.0: A scalable data generator and benchmark with strong domain randomization for robust bimanual robotic manipulation," *arXiv preprint arXiv:2506.18088*, 2025.

[14] T. Chen, Y. Chen, Z. Li, J. Tang, K. Su, H. Lu, W. Wan, B. Chen, S. Liu, H. Yan *et al.*, "RoboDojo: A unified sim-and-real benchmark for comprehensive evaluation of generalist robot manipulation policies," *arXiv preprint arXiv:2607.04434*, 2026.

[15] D. Driess, F. Xia, M. S. M. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, W. Huang, Y. Chebotar, P. Sermanet, D. Duckworth, S. Levine, V. Vanhoucke, K. Hausman, M. Toussaint, K. Greff, A. Zeng, I. Mordatch, and P. Florence, "Palm-e: An embodied multimodal language model," *arXiv preprint arXiv:2303.03378*, 2023. [Online]. Available: https://arxiv.org/abs/2303.03378

[16] X. Li, M. Liu, H. Zhang, C. Yu, J. Xu, H. Wu, C. Cheang, Y. Jing, W. Zhang, H. Liu, H. Li, and T. Kong, "Vision-language foundation models as effective robot imitators," in *International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=lFYj0oibGR

[17] Open X-Embodiment Collaboration, A. O'Neill, A. Rehman, A. Gupta, A. Maddukuri, A. Gupta, A. Padalkar, A. Jain, A. Bewley, K. Hausman, S. Levine, C. Finn *et al.*, "Open x-embodiment: Robotic learning datasets and rt-x models," *arXiv preprint arXiv:2310.08864*, 2023. [Online]. Available: https://arxiv.org/abs/2310.08864

[18] Octo Model Team, D. Ghosh, H. Walke, K. Pertsch, K. Black, O. Mees, S. Dasari, J. Hejna, T. Kreiman, C. Xu, Y. L. Tan, L. Y. Chen, P. Sanketi, Q. Vuong, T. Xiao, D. Sadigh, C. Finn, and S. Levine, "Octo: An open-source generalist robot policy," *arXiv preprint arXiv:2405.12213*, 2024. [Online]. Available: https://arxiv.org/abs/2405.12213

[19] Q. Zhao, Y. Lu, M. J. Kim, Z. Fu, Z. Zhang, Y. Wu, Z. Li, Q. Ma, S. Han, C. Finn, A. Handa, M.-Y. Liu, D. Xiang, G. Wetzstein, and T.-Y. Lin, "Cot-vla: Visual chain-of-thought reasoning for vision-language-action models," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. [Online]. Available: https://arxiv.org/abs/2503.22020

[20] H. Wang, X. Wei, J. He, C. Bai, C.-K. Fan, J. Cao, J. Chen, Y. Li, S. Rong, M. Lu, X. Ju, J. Tang, and S. Zhang, "VEGA: Visual Encoder Grounding Alignment for Spatially-Aware Vision-Language-Action Models," *arXiv preprint arXiv:2605.10485*, 2026. [Online]. Available: https://arxiv.org/abs/2605.10485

[21] S. Belkhale, T. Ding, T. Xiao, P. Sermanet, Q. Vuong, J. Tompson, Y. Chebotar, D. Dwibedi, and D. Sadigh, "Rt-h: Action hierarchies using language," in *Robotics: Science and Systems XX*, 2024. [Online]. Available: https://arxiv.org/abs/2403.01823

[22] W. Huang, C. Wang, R. Zhang, Y. Li, J. Wu, and L. Fei-Fei, "VoxPoser: Composable 3D value maps for robotic manipulation with language models," in *Proceedings of The 7th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 229. PMLR, 2023, pp. 540–562. [Online]. Available: https://proceedings.mlr.press/v229/huang23b.html

[23] B. Lin, Y. Nie, Z. Wei, J. Chen, S. Ma, J. Han, H. Xu, X. Chang, and X. Liang, "NavCoT: Boosting LLM-Based Vision-and-Language Navigation via Learning Disentangled Reasoning," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, no. 7, pp. 5945–5957, 2025. [Online]. Available: https://arxiv.org/abs/2403.07376

[24] Y. Xu, Y. Pan, and Z. Liu, "Dream to Recall: Imagination-Guided Experience Retrieval for Memory-Persistent Vision-and-Language Navigation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2026. [Online]. Available: https://arxiv.org/abs/2510.08553

[25] F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine, "Visual foresight: Model-based deep reinforcement learning for vision-based robotic control," *arXiv preprint arXiv:1812.00568*, 2018. [Online]. Available: https://arxiv.org/abs/1812.00568

[26] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=S1lOTC4tDS

[27] G. Zhou, H. Pan, Y. LeCun, and L. Pinto, "Dino-wm: World models on pre-trained visual features enable zero-shot planning," *arXiv preprint arXiv:2411.04983*, 2024. [Online]. Available: https://arxiv.org/abs/2411.04983

[28] M. Assran, A. Bardes, D. Fan, Q. Garrido, R. Howes, M. Komeili, M. Muckley, A. Rizvi, C. Roberts, K. Sinha, A. Zholus, S. Arnaud, A. Gejji, A. Martin, F. R. Hogan, D. Dugas, P. Bojanowski, V. Khalidov, P. Labatut, F. Massa, M. Szafraniec, K. Krishnakumar, Y. Li, X. Ma, S. Chandar, F. Meier, Y. LeCun, M. Rabbat, and N. Ballas, "V-jepa 2: Self-supervised video models enable understanding, prediction

and planning," *arXiv preprint arXiv:2506.09985*, 2025. [Online]. Available: https://arxiv.org/abs/2506.09985

[29] A. Bar, G. Zhou, D. Tran, T. Darrell, and Y. LeCun, "Navigation world models," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2025. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2025/html/Bar_Navigation_World_Models_CVPR_2025_paper.html

[30] J. Jang, S. Ye, Z. Lin, J. Xiang, J. Bjorck, Y. Fang, F. Hu, S. Huang, K. Kundalia, Y.-C. Lin, L. Magne, A. Mandlekar, A. Narayan, Y. L. Tan, G. Wang, J. Wang, Q. Wang, Y. Xu, X. Zeng, K. Zheng, R. Zheng, M.-Y. Liu, L. Zettlemoyer, D. Fox, J. Kautz, S. Reed, Y. Zhu, and L. Fan, "Dreamgen: Unlocking generalization in robot learning through video world models," in *Proceedings of the 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 5170–5194. [Online]. Available: https://proceedings.mlr.press/v305/jang25a.html

[31] J. Liang, P. Tokmakov, R. Liu, S. Sudhakar, P. Shah, R. Ambrus, and C. Vondrick, "Video generators are robot policies," *arXiv preprint arXiv:2508.00795*, 2025. [Online]. Available: https://arxiv.org/abs/2508.00795

[32] Y. Liao, P. Zhou, S. Huang, D. Yang, S. Chen, Y. Jiang, Y. Hu, J. Cai, S. Liu, J. Luo, L. Chen, S. Yan, M. Yao, and G. Ren, "Genie envisioner: A unified world foundation platform for robotic manipulation," in *International Conference on Learning Representations*, 2026. [Online]. Available: https://openreview.net/forum?id=fHLtSxDFKC

[33] Q. Zhang *et al.*, "Native video-action pretraining for generalizable robot control," *arXiv preprint arXiv:2607.08639*, 2026. [Online]. Available: https://arxiv.org/abs/2607.08639

[34] Y. Li, X. Wei, J. Cao, H. Wang, X. Chi, C. Bai, Q. Sun, J. Li, X. Zhang, P. Jia, J. Tang, S. Han, and S. Zhang, "WAM4D: Fast 4D World Action Model via Spatial Register Tokens," *arXiv preprint arXiv:2606.14048*, 2026. [Online]. Available: https://arxiv.org/abs/2606.14048

[35] S. Chen, J. Liu, J. Wu, Y. Guo, T. Wang, S. Qian, H. Chen, J. Cao, J. Tang, and S. Zhang, "Do Robotic World Models Really Follow Actions? Diagnosing and Aligning Action-Conditioned Generation for Policy Learning," *arXiv preprint arXiv:2608.24885*, 2026. [Online]. Available: https://arxiv.org/abs/2608.24885

[36] M. J. Kim, Y. Gao, T.-Y. Lin, Y.-C. Lin, Y. Ge, G. Lam, P. Liang, S. Song, M.-Y. Liu, C. Finn, and J. Gu, "Cosmos Policy: Fine-Tuning Video Models for Visuomotor Control and Planning," *arXiv preprint arXiv:2601.16163*, 2026. [Online]. Available: https://arxiv.org/abs/2601.16163

[37] J. Chen, P. Jia, Q. Wuwu, J. Liu, M. Du, C.-K. Fan, X. Chi, H. Chen, C. Bai, Z. Qian, H. Wang, J. Cao, W. Mi, X. Ju, J. Tang, and S. Zhang, "MV-WAM: Manifold-Aware World Action Model with Value Augmentation," *arXiv preprint arXiv:2606.21088*, 2026. [Online]. Available: https://arxiv.org/abs/2606.21088

[38] K. Tang, P. Jia, Z. Chu, J. Wu, R. Ma, J. Cao, F. Zhao, S. Chen, Y. Guo, X. Chi, C.-K. Fan, K. Zhang, J. Xu, F. Yang, W. Mi, X. Ju, J. Tang, and S. Zhang, "SafeDojo: Safe Reinforcement Learning for VLA via Interactive World Model," *arXiv preprint arXiv:2606.20698*, 2026. [Online]. Available: https://arxiv.org/abs/2606.20698

[39] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," *arXiv preprint arXiv:2106.09685*, 2021.

[40] J. Farebrother, J. Orbay, Q. Vuong, A. A. Taïga, Y. Chebotar, T. Xiao, A. Irpan, S. Levine, P. S. Castro, A. Faust *et al.*, "Stop regressing: Training value functions via classification for scalable deep rl," *arXiv preprint arXiv:2403.03950*, 2024.

[41] F. L. Markley, Y. Cheng, J. L. Crassidis, and Y. Oshman, "Averaging quaternions," *Journal of Guidance, Control, and Dynamics*, vol. 30, no. 4, pp. 1193–1197, 2007.

[42] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge *et al.*, "Qwen3-VL technical report," *arXiv preprint arXiv:2511.21631*, 2025.

[43] J. Zheng, J. Li, Z. Wang, D. Liu, X. Kang, Y. Feng, Y. Zheng, J. Zou, Y. Chen, J. Zeng, Y.-Q. Zhang, J. Pang, J. Liu, T. Wang, and X. Zhan, "X-VLA: Soft-prompted transformer as scalable cross-embodiment vision-language-action model," *arXiv preprint arXiv:2510.10274*, 2025. [Online]. Available: https://arxiv.org/abs/2510.10274

[44] K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. R. Equi, C. Finn, N. Fusai, M. Y. Galliker, D. Ghosh, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, D. LeBlanc, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, A. Z. Ren, L. X. Shi, L. Smith, J. T. Springenberg, K. Stachowicz, J. Tanner, Q. Vuong, H. Walke, A. Walling, H. Wang, L. Yu, and U. Zhilinsky, "$\pi_{0.5}$: A vision-language-action model with open-world generalization," in *Proceedings of The 9th Conference on Robot Learning*, ser. Proceedings of Machine Learning Research, vol. 305. PMLR, 2025, pp. 17–40. [Online]. Available: https://proceedings.mlr.press/v305/black25a.html

[45] H. Yuan, Z. Liang, A. Chen, Y. Wang, H. Li, P. Lin, Y. Huang, Z. Lei, T. Zhang, J. Zhang, J. Zhang, J. Fan, G. Zhou, Q. Peng, C. Lv, X. Chen, A. Yang, F. Huang, J. Lin, D. Liu, J. Zhou, C. Wu, and X.-H. Chen, "Qwen-RobotManip technical report: Alignment unlocks scale for robotic manipulation foundation models," *arXiv preprint arXiv:2606.17846*, 2026. [Online]. Available: https://arxiv.org/abs/2606.17846

[46] H. Bi, H. Tan, S. Xie, Z. Wang, S. Huang, H. Liu, R. Zhao, Y. Feng, C. Xiang, Y. Rong, H. Zhao, H. Liu, Z. Su, L. Ma, H. Su, and J. Zhu, "Motus: A unified latent action world model," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2026, pp. 35 101–35 113. [Online]. Available: https://openaccess.thecvf.com/content/CVPR2026/html/Bi_Motus_A_Unified_Latent_Action_World_Model_CVPR_2026_paper.html

[47] A. Ye, B. Wang, C. Ni, G. Huang, G. Zhao, H. Li, H. Li, J. Li, J. Lv, J. Liu, M. Cao, P. Li, Q. Deng, W. Mei, X. Wang, X. Chen, X. Zhou, Y. Wang, Y. Chang, Y. Li, Y. Zhou, Y. Ye, Z. Liu, and Z. Zhu, "GigaWorld-Policy: An efficient action-centered world–action model," *arXiv preprint arXiv:2603.17240*, 2026. [Online]. Available: https://arxiv.org/abs/2603.17240

[48] J. Lyu, K. Liu, X. Zhang, H. Liao, Y. Feng, W. Zhu, T. Shen, J. Chen, J. Zhang, Y. Dong, W. Cui, S. Qi, S. Wang, Y. Zheng, M. Yan, X. Shi, H. Li, D. Zhao, M.-Y. Liu, Z. Zhang, Y. Li, Y. Wang, and H. Wang, "LDA-1B: Scaling latent dynamics action model via universal embodied data ingestion," *arXiv preprint arXiv:2602.12215*, 2026. [Online]. Available: https://arxiv.org/abs/2602.12215

[49] S. Liu, L. Wu, B. Li, H. Tan, H. Chen, Z. Wang, K. Xu, H. Su, and J. Zhu, "RDT-1B: A diffusion foundation model for bimanual manipulation," *arXiv preprint arXiv:2410.07864*, 2024. [Online]. Available: https://arxiv.org/abs/2410.07864

[50] H. Bi, L. Wu, T. Lin, H. Tan, Z. Su, H. Su, and J. Zhu, "H-RDT: Human manipulation enhanced bimanual robotic manipulation," *arXiv preprint arXiv:2507.23523*, 2025. [Online]. Available: https://arxiv.org/abs/2507.23523

[51] J. Guo, Q. Li, P. Li, Z. Chen, N. Sun, Y. Su, H. Wang, Y. Zhang, X. Li, and H. Liu, "Unified 4d world action modeling from video priors with asynchronous denoising," *arXiv preprint arXiv:2604.26694*, 2026. [Online]. Available: https://arxiv.org/abs/2604.26694

[52] Z. Qian, X. Chi, C.-W. Mak, T. Zhou, R. Yuan, Y. Rui, H. Sun, Z. Wu, Y. Li, S. Qian, S. Han, and S. Zhang, "Hierarchical denoising for multi-step visual reasoning," *arXiv preprint arXiv:2607.15278*, 2026. [Online]. Available: https://arxiv.org/abs/2607.15278

[53] J. Cai, L. Ling, S. Chu, Z. Liu, J. Kang, Z. Liang, W. Xu, Y. Mao, W. Zhang, X. Yang, R. Ying, R. Zheng, and Y. Mu, "AHA-WAM: Asynchronous horizon-adaptive world-action modeling with observation-guided context routing," *arXiv preprint arXiv:2606.09811*, 2026. [Online]. Available: https://arxiv.org/abs/2606.09811

[54] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning fine-grained bimanual manipulation with low-cost hardware," *arXiv preprint arXiv:2304.13705*, 2023. [Online]. Available: https://arxiv.org/abs/2304.13705

[55] Y. Wang, S. Huang, M. Li, C. Zhang, J. Liang, W. Jin, Y. Chen, X. Chi, D. Zhou, Q. Yu *et al.*, "Openwam: An open, modular exploration towards systematic world–action model pretraining," *arXiv preprint arXiv:2609.07398*, 2026. [Online]. Available: https://arxiv.org/abs/2609.07398

[56] G. Yan, J. Liu, Y. Fan, L. Cai, M. Liao, J. Zhang, and D. Fox, "Flex-*pi*: A multi-stream world-action model with compute flexibility," *arXiv preprint arXiv:2608.10860*, 2026. [Online]. Available: https://arxiv.org/abs/2608.10860

[57] Motubrain Team, C. Xiang, F. Bao, H. Liu, H. Tan, H. Bi, J. Li, J. Liu, J. Pang, K. Jing, L. Liu, M. Cai, R. Cui, R. Zhao, R. Wang, S. Huang, Y. Feng, Y. Rong, Z. Wang, and J. Zhu, "MotuBrain: An advanced world action model for robot control," *arXiv preprint arXiv:2604.27792*, 2026. [Online]. Available: https://arxiv.org/abs/2604.27792

APPENDIX A
PER-TASK ROBOTWIN RESULTS

Table 12 reports success rates on all 50 RoboTwin tasks for LAW with Fast-WAM and OpenWAM-α backbones, together with the compared baselines. Baseline entries reproduce reported per-task results. Means are computed with equal weight per task and, for the overall score, per evaluation setting. LAW + Fast-WAM achieves 94.22% and 94.20% on the clean and randomized settings. LAW + OpenWAM-α achieves 95.60% and 95.52%, respectively, yielding an overall average of 95.56%. The OpenWAM-based variant uses 100 evaluation episodes per task and setting and corresponds to the result in Table 11.

TABLE 12
Per-task results on the 50-task RoboTwin benchmark.

Task	LAW + OpenWAM-α		LAW + Fast-WAM		Fast-WAM		LingBot-VA		$\pi_{0.5}$	
	Clean (%)	Rand. (%)	Clean (%)	Rand. (%)	Clean (%)	Rand. (%)	Clean (%)	Rand. (%)	Clean (%)	Rand. (%)
Adjust Bottle	**100**	**100**	**100**	**100**	**100**	**100**	90	94	**100**	99
Beat Block Hammer	97	**100**	96	99	**99**	97	96	98	96	93
Blocks Ranking RGB	**100**	99	95	93	**100**	**100**	99	98	92	85
Blocks Ranking Size	91	91	79	75	**94**	**98**	**94**	96	49	26
Click Alarmclock	99	99	**100**	99	**100**	**100**	99	**100**	98	89
Click Bell	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99	66
Dump Bin Bigbin	95	91	94	**97**	**97**	96	89	96	92	**97**
Grab Roller	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**
Handover Block	**100**	**97**	**100**	**97**	95	81	99	78	66	57
Handover Mic	**99**	94	97	94	**99**	**100**	94	96	98	97
Hanging Mug	**77**	**78**	44	55	58	62	40	28	18	17
Lift Pot	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99	96	85
Move Can Pot	**100**	**99**	**100**	98	90	88	94	97	51	55
Move Pillbottle Pad	99	99	**100**	**100**	**100**	99	99	99	84	61
Move Playingcard Away	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99	96	84
Move Stapler Pad	**98**	**96**	86	85	77	64	91	79	56	42
Open Laptop	99	98	**100**	99	98	**100**	92	94	90	96
Open Microwave	**97**	**96**	96	95	62	45	82	86	34	77
Pick Diverse Bottles	**93**	**93**	90	90	80	85	89	82	81	71
Pick Dual Bottles	**100**	**100**	**100**	**100**	**100**	96	**100**	99	93	63
Place A2B Left	94	**98**	**99**	96	95	93	97	93	87	82
Place A2B Right	**99**	**100**	98	97	93	99	97	95	87	84
Place Bread Basket	96	**96**	**99**	95	91	93	97	95	77	64
Place Bread Skillet	**95**	94	94	**95**	90	93	**95**	90	85	66
Place Burger Fries	95	97	**99**	**99**	96	**99**	97	95	94	87
Place Can Basket	**93**	**86**	79	84	71	69	81	84	62	62
Place Cans Plasticbox	**100**	99	**100**	**100**	99	96	**100**	99	94	84
Place Container Plate	96	99	96	98	96	**100**	**99**	97	**99**	95
Place Dual Shoes	**99**	**98**	95	96	94	88	94	89	75	75
Place Empty Cup	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99
Place Fan	**99**	95	96	95	96	**96**	**99**	93	87	85
Place Mouse Pad	**100**	**99**	95	96	83	89	93	96	60	39
Place Object Basket	88	**92**	**93**	91	89	88	91	88	80	76
Place Object Scale	**99**	**98**	90	96	90	97	96	95	86	80
Place Object Stand	**100**	**100**	98	**100**	90	94	99	96	91	85
Place Phone Stand	97	96	**99**	96	97	**99**	97	97	81	81
Place Shoe	99	**100**	**100**	**100**	96	99	98	98	92	93
Press Stapler	93	94	**94**	96	90	**97**	85	82	87	83
Put Bottles Dustbin	93	**97**	92	96	**95**	90	87	91	84	79
Put Object Cabinet	87	85	91	**89**	**94**	**89**	85	87	80	79
Rotate QRcode	94	**95**	**97**	93	93	89	96	91	89	87
Scan Object	**97**	**92**	96	87	89	**92**	96	91	72	65
Shake Bottle	**100**	**100**	**100**	**100**	**100**	**100**	**100**	97	99	97
Shake Bottle Horizontally	**100**	**100**	**100**	**100**	**100**	**100**	**100**	99	99	99
Stack Blocks Three	97	97	98	97	95	97	**99**	**98**	91	76
Stack Blocks Two	**100**	**100**	**100**	**100**	**100**	**100**	**100**	98	97	**100**
Stack Bowls Three	80	80	**89**	**87**	80	81	86	83	77	71
Stack Bowls Two	**96**	97	92	96	92	**98**	94	**98**	95	96
Stamp Seal	**100**	**100**	**100**	99	90	94	96	97	79	55
Turn Switch	50	**62**	55	60	61	59	44	45	**62**	54
Average	**95.60**	**95.52**	94.22	94.20	91.88	91.78	92.90	91.50	82.74	76.76

APPENDIX B
REAL-WORLD EXPERIMENTAL DETAILS

B.1 Robot Platform and Observations

Experiments are conducted on an AgileX (Songling) Cobot Magic platform equipped with two six-degree-of-freedom PiPER arms and parallel-jaw grippers. Figure 10 illustrates the hardware platform. Manipulation takes place in a fixed tabletop workspace. Visual observations comprise three RGB views: a scene camera covering the shared workspace and a wrist-mounted camera on each arm. The scene view provides task-level context, while the wrist views resolve local interactions between the grippers, objects, and target receptacles. Robot placement and camera mounting geometry are held fixed during data collection to maintain a consistent observation configuration.



Fig. 10. AgileX Cobot Magic platform. Two follower arms perform bimanual manipulation, while two leader arms support demonstration collection. The observation configuration comprises one scene camera and two wrist cameras. Hardware reference image from AgileX Robotics.

B.2 Demonstration Collection and Data Representation

Human demonstrations are collected through teleoperation. Across episodes, object positions and in-plane orientations are varied within the reachable workspace, subject to the geometric constraints of each task. This variation changes the required grasps and placements while preserving the task objective. Each trajectory associates multiview visual observations with bimanual robot states, action sequences, a language instruction, and an episode outcome. Semantic annotations specify the subtask descriptions and their temporal boundaries, allowing the same trajectory to support action learning and supervision of task-level planning and state estimation.

The collection protocol retains successful, partially completed, and failed executions. Failures are annotated by the stage at which they occur and their principal cause, preserving information about completed task prefixes. Recording validity is assessed separately from task success: episodes with missing visual or robot data, unusable temporal alignment, or severe occlusion of the manipulation are excluded. No fixed success-to-failure ratio is imposed during collection.

Trajectories are organized in LeRobot format with episode-level metadata and aligned camera streams. Recorded videos are stored at 30 frames per second. LAW uses the end-effector action representation described in the main paper: a three-dimensional position, a unit quaternion, and a scalar gripper command for each arm, giving a 16-dimensional bimanual action vector. Semantic boundaries are indexed in the trajectory timeline so that visual observations, actions, and active-subtask labels refer to the same execution interval.

B.3 Task Definitions

The evaluation comprises three tasks with complementary requirements for sequential manipulation, spatial alignment, and state-dependent interaction.

Domino. The robot arranges four dominoes into an upright row and initiates their toppling. Successful execution requires stable placement and sufficient spatial alignment for the intended toppling sequence. The task contains five semantic stages: four individual placements and the final toppling action.

Puzzle. The robot inserts three colored pieces into their corresponding recesses in a puzzle board. Each piece constitutes one semantic stage. The task requires identifying the matching target and aligning the piece with the recess before insertion; success requires all three pieces to be correctly seated.

Organize. The robot sorts three colored blocks into the corresponding drawers of a three-level cabinet. For each block, the robot opens the target drawer, places the block inside, and closes the drawer, resulting in nine semantic stages. Success requires correct block–drawer assignments and closure of all drawers. This task combines object manipulation with repeated changes in the state of an articulated receptacle.

B.4 Evaluation Metrics

We distinguish complete-task success from partial progress. For a task with N evaluation episodes and K semantic stages, let $y_i \in \{0,1\}$ indicate whether episode i satisfies the complete task objective, and let $c_{i,k} \in \{0,1\}$ indicate completion of stage k. We report

$$\mathrm{SR} = \frac{100}{N}\sum_{i=1}^{N} y_i, \qquad \mathrm{Score} = \frac{100}{NK}\sum_{i=1}^{N}\sum_{k=1}^{K} c_{i,k}. \tag{19}$$

Thus, the progress score assigns equal weight to the semantic stages within each task, with $K = 5$, 3, and 9 for Domino, Puzzle, and Organize, respectively. A partially completed execution can receive a nonzero progress score while remaining unsuccessful under the complete-task criterion. This distinction captures intermediate progress in long-horizon execution without conflating it with achievement of the final objective.

APPENDIX C
SEMANTIC SUBTASK DECOMPOSITION

Tables 13 and 14 expand the examples in Fig. 3 with complete task inventories. Each subtask specifies a semantic objective and its associated trajectory interval. Grasping, transport, and release can form one placement stage, whereas handovers and explicit confirmation actions can define separate stages. Repeated motions serving the same objective may remain within one subtask; object identities and spatial relations refer to the corresponding task instance.

TABLE 13
Semantic subtask sequences for 50 RoboTwin tasks.

Task	Ordered semantic subtasks
Adjust Bottle	**(1)** Pick up the target bottle and hold it upright.
Beat Block Hammer	**(1)** Hit the block with the hammer.
Blocks Ranking RGB	**(1)** Place the red block at the left position; **(2)** Place the green block at the middle position; **(3)** Place the blue block at the right position.
Blocks Ranking Size	**(1)** Place the small block at the right position; **(2)** Place the medium block at the middle position; **(3)** Place the large block at the left position.
Click Alarmclock	**(1)** Press the alarm clock button.
Click Bell	**(1)** Press the top of the bell.
Dump Bin Bigbin	**(1)** Pour the trash bin contents into the big bin.
Grab Roller	**(1)** Lift the roller.
Handover Block	**(1)** Pick up the red block; **(2)** Transfer the red block to the other hand; **(3)** Place the red block on the blue pad.
Handover Mic	**(1)** Pick up the microphone; **(2)** Transfer the microphone to the other hand.
Hanging Mug	**(1)** Place the mug at the center of the table; **(2)** Place the mug on the rack.
Lift Pot	**(1)** Lift the pot off the table.
Move Can Pot	**(1)** Place the can next to the pot.
Move Pillbottle Pad	**(1)** Place the pill bottle on the pad.
Move Playingcard Away	**(1)** Move the playing-card box outward.
Move Stapler Pad	**(1)** Place the stapler on the target mat.
Open Laptop	**(1)** Open the laptop lid.
Open Microwave	**(1)** Open the microwave door to more than 70 degrees.
Pick Diverse Bottles	**(1)** Pick up the two target bottles.
Pick Dual Bottles	**(1)** Pick up the two target bottles.
Place A2B Left	**(1)** Place the target object to the left of the reference object.
Place A2B Right	**(1)** Place the target object to the right of the reference object.
Place Bread Basket	**(1)** Place the bread in the breadbasket.
Place Bread Skillet	**(1)** Place the bread in the skillet.
Place Burger Fries	**(1)** Place the hamburger on the tray; **(2)** Place the french fries on the tray.
Place Can Basket	**(1)** Place the can in the basket; **(2)** Lift the basket 5 cm above its initial height.
Place Cans Plasticbox	**(1)** Place the left can in the plastic box; **(2)** Place the right can in the plastic box.
Place Container Plate	**(1)** Place the container on the plate.
Place Dual Shoes	**(1)** Place the left shoe in the shoe box; **(2)** Place the right shoe in the shoe box.
Place Empty Cup	**(1)** Place the empty cup on the coaster.
Place Fan	**(1)** Place the fan on the target mat facing the robot.
Place Mouse Pad	**(1)** Place the mouse on the target mat.
Place Object Basket	**(1)** Place the object in the basket; **(2)** Lift the basket.
Place Object Scale	**(1)** Place the target object on the electronic scale.
Place Object Stand	**(1)** Place the target object on the display stand.
Place Phone Stand	**(1)** Place the phone on the phone stand.
Place Shoe	**(1)** Place the shoe on the mat.
Press Stapler	**(1)** Press the stapler.
Put Bottles Dustbin	**(1)** Put one bottle in the dustbin; **(2)** Put another bottle in the dustbin; **(3)** Put the remaining bottle in the dustbin.
Put Object Cabinet	**(1)** Grasp and lift the object; **(2)** Pull the cabinet drawer open; **(3)** Place the object near the center inside the cabinet drawer and fully release it.
Rotate QRcode	**(1)** Pick up the payment sign; **(2)** Rotate the payment sign until the QR code faces the robot.
Scan Object	**(1)** Pick up the scanner and the object; **(2)** Scan the object with the scanner.
Shake Bottle	**(1)** Pick up the bottle; **(2)** Shake the bottle.
Shake Bottle Horizontally	**(1)** Pick up the bottle; **(2)** Shake the bottle horizontally.
Stack Blocks Three	**(1)** Place the red block at the center of the table as the base; **(2)** Stack the green block on the red block; **(3)** Stack the blue block on the green block.
Stack Blocks Two	**(1)** Place the red block at the center of the table as the base; **(2)** Stack the green block on the red block.
Stack Bowls Three	**(1)** Place one bowl as the base; **(2)** Stack another bowl inside the base bowl; **(3)** Stack the remaining bowl inside the stacked bowls.
Stack Bowls Two	**(1)** Place one bowl as the base; **(2)** Stack the remaining bowl inside the base bowl.
Stamp Seal	**(1)** Stamp the target area with the seal.
Turn Switch	**(1)** Activate the switch.

TABLE 14
Semantic subtask sequences for 35 RoboDojo tasks.

Task	Ordered semantic subtasks
Arrange Largest Number	**(1)** Arrange the digits from left to right to form the largest number.
Build Tower	**(1)** Place the two base blocks of the first tower layer; **(2)** Place the first-layer beam across the base blocks; **(3)** Place the two base blocks of the second tower layer; **(4)** Place the second-layer beam across the second-layer base; **(5)** Place the cuboid block on the tower; **(6)** Place the pyramid on top to complete the tower.
Classify Objects	**(1)** Sort objects into the white basket; **(2)** Sort objects into the blue basket; **(3)** Sort objects into the red basket.
Cover Blocks	**(1)** Cover the leftmost block; **(2)** Cover the middle block; **(3)** Cover the rightmost block; **(4)** Uncover the red block; **(5)** Uncover the green block; **(6)** Uncover the blue block.
Deposit Coin	**(1)** Pick up the coin; **(2)** Hand the coin to the other hand; **(3)** Insert the coin into the coin bank.
DLC	**(1)** Arrange the letters to spell "RoboDojo" in a row.
Fasten Screws	**(1)** Insert the first screw and tighten it with five turns; **(2)** Insert the second screw and tighten it with five turns; **(3)** Insert the third screw and tighten it with five turns.
Fill Egg Holder	**(1)** Place all eggs into the egg holder; **(2)** Close the egg-holder lid.
Fill Pen Holder	**(1)** Pick up the pen holder; **(2)** Place all pens into the pen holder; **(3)** Put the pen holder back down on the table.
Fold Clothes	**(1)** Fold the left sleeve; **(2)** Fold the right sleeve; **(3)** Fold the hem.
Hang Mugs	**(1)** Hang all mugs on the mug rack.
Imitate Sorting Sequence	**(1)** Observe and remember the demonstrated placement sequence; **(2)** Place all objects following the demonstrated sequence.
Insert Key	**(1)** Pick up the key; **(2)** Hand the key to the other hand; **(3)** Insert the key into the keyhole and turn it.
Insert Tubes	**(1)** Insert all tubes into the rack.
Make Kong	**(1)** Wait for the opponent to discard a matching tile; **(2)** Declare a kong with the matching tiles; **(3)** Draw a replacement tile.
Make Toast	**(1)** Place both bread slices into the toaster; **(2)** Press the toaster lever down.
Match And Pick From Conveyor	**(1)** Observe and remember the first object on the conveyor; **(2)** Pick the matching object when it reappears.
Organize Table	**(1)** Place the alarm clock on the drawer; **(2)** Place the figurine on the stand; **(3)** Place the mouse on the mouse pad; **(4)** Push the keyboard into the frame.
Pack Objects Into Box	**(1)** Pack the objects into the box with their front sides facing left.
Play Xylophone	**(1)** Pick up the mallet; **(2)** Strike all xylophone keys from left to right.
Play Stacking Toy	**(1)** Place all pieces of the first stacking toy onto their peg; **(2)** Place all pieces of the second stacking toy onto their peg; **(3)** Place all pieces of the third stacking toy onto their peg; **(4)** Place all pieces of the fourth stacking toy onto their peg.
Play Tic Tac Toe	**(1)** Place all robot marks on the tic-tac-toe board.
Plug In Charger	**(1)** Pick up the charger; **(2)** Hand the charger to the other hand; **(3)** Plug the charger into the power strip.
Pour Balls Into Vase	**(1)** Pour all balls from the cup into the vase.
Pour Liquid Into Cup	**(1)** Pour the liquid from the bottle into the cup.
Press By Number	**(1)** Press the first red button the required number of times; **(2)** Press the blue button to confirm; **(3)** Press the second red button the required number of times; **(4)** Press the blue button to confirm.
Push T	**(1)** Push the T-shaped block into alignment with the gray T-shaped pad.
Put Bottles Into Dustbin	**(1)** Put all bottles into the dustbin.
Sort Nesting Dolls By Size	**(1)** Place all nesting dolls in size order from left to right, smallest to largest.
Stack Blocks	**(1)** Place the first block as the tower base; **(2)** Stack the second block on the base; **(3)** Stack the final block to complete the tower.
Stack Bowls	**(1)** Stack the three bowls together.
Store Laptop And Headphones	**(1)** Hang the headphones on the headphone stand; **(2)** Close the laptop; **(3)** Place the laptop into the vertical laptop stand.
Swap T	**(1)** Pick up the first T-shaped block; **(2)** Pick up the second T-shaped block; **(3)** Place the first T-shaped block at the second T's original position; **(4)** Place the second T-shaped block at the first T's original position.
Swap Blocks	**(1)** Move the first block to the empty mat; **(2)** Press the button after moving the first block; **(3)** Move the second block to the first block's original position; **(4)** Press the button after moving the second block; **(5)** Move the first block to the second block's original position; **(6)** Press the button to complete the swap.
Sweep Blocks	**(1)** Pick up the broom; **(2)** Hand the broom to the right hand; **(3)** Sweep the blocks into the dustpan.

C.1 Real-World Subtask Sequences

Table 15 specifies the five, three, and nine stages for Domino, Puzzle, and Organize, respectively, consistent with the task definitions and progress metric in Appendix B.

TABLE 15
Semantic subtask sequences for real-world tasks.

Task	Ordered semantic subtasks
Domino	**(1–4)** Place the first, second, third, and fourth dominoes upright in sequence; **(5)** initiate the toppling sequence.
Puzzle	**(1)** Insert the first piece into its matching recess; **(2)** insert the second piece; **(3)** insert the third piece.
Organize	**(1)** Open the drawer for block 1; **(2)** place block 1 inside; **(3)** close the drawer; **(4)** open the drawer for block 2; **(5)** place block 2 inside; **(6)** close the drawer; **(7)** open the drawer for block 3; **(8)** place block 3 inside; **(9)** close the drawer.

C.2 Segmented Trajectory Examples

Figures 11–14 provide additional examples from the simulation datasets. Each row corresponds to a task, and the labeled groups delineate semantic subtasks in execution order from left to right. One or two representative observations depict each stage. The grouping follows changes in the semantic objective rather than uniform temporal sampling.

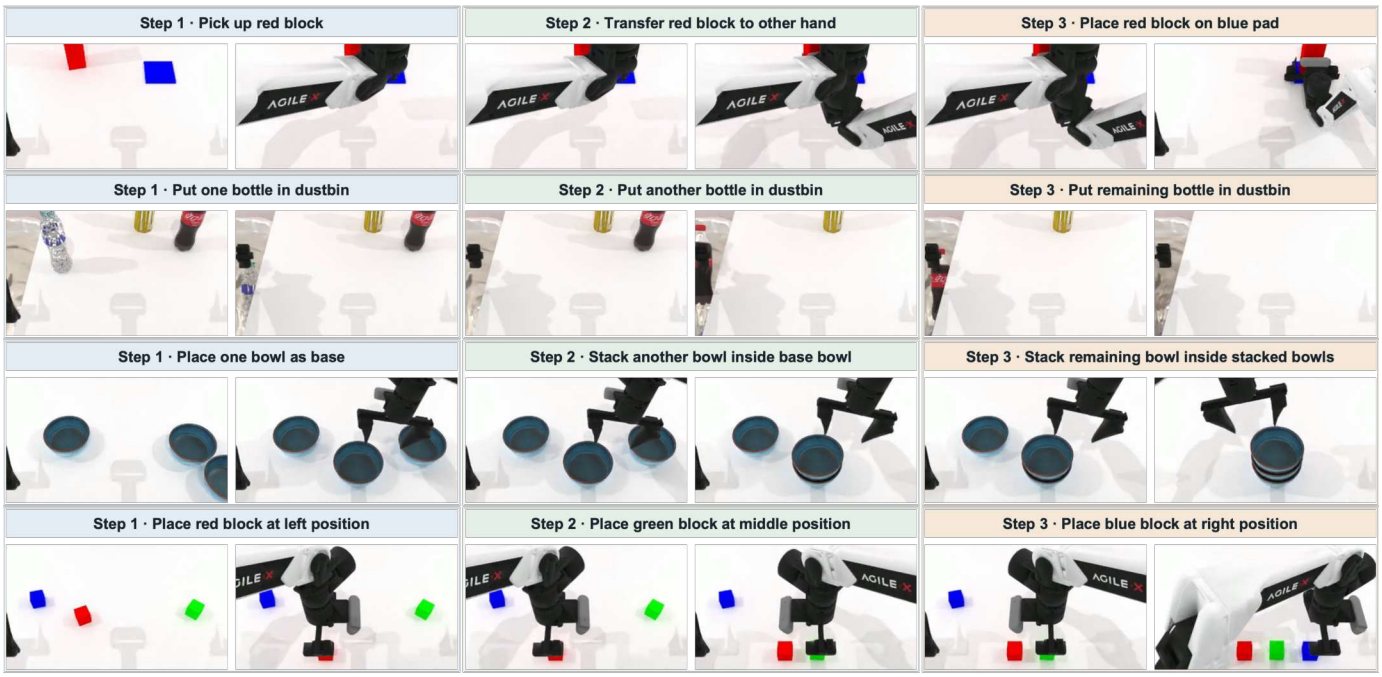


Fig. 11. Semantic subtask segmentation on RoboTwin. From top to bottom: Handover Block, Put Bottles Dustbin, Stack Bowls Three, and Blocks Ranking RGB. The examples distinguish acquisition, inter-arm transfer, and placement, as well as successive placements involving different objects.

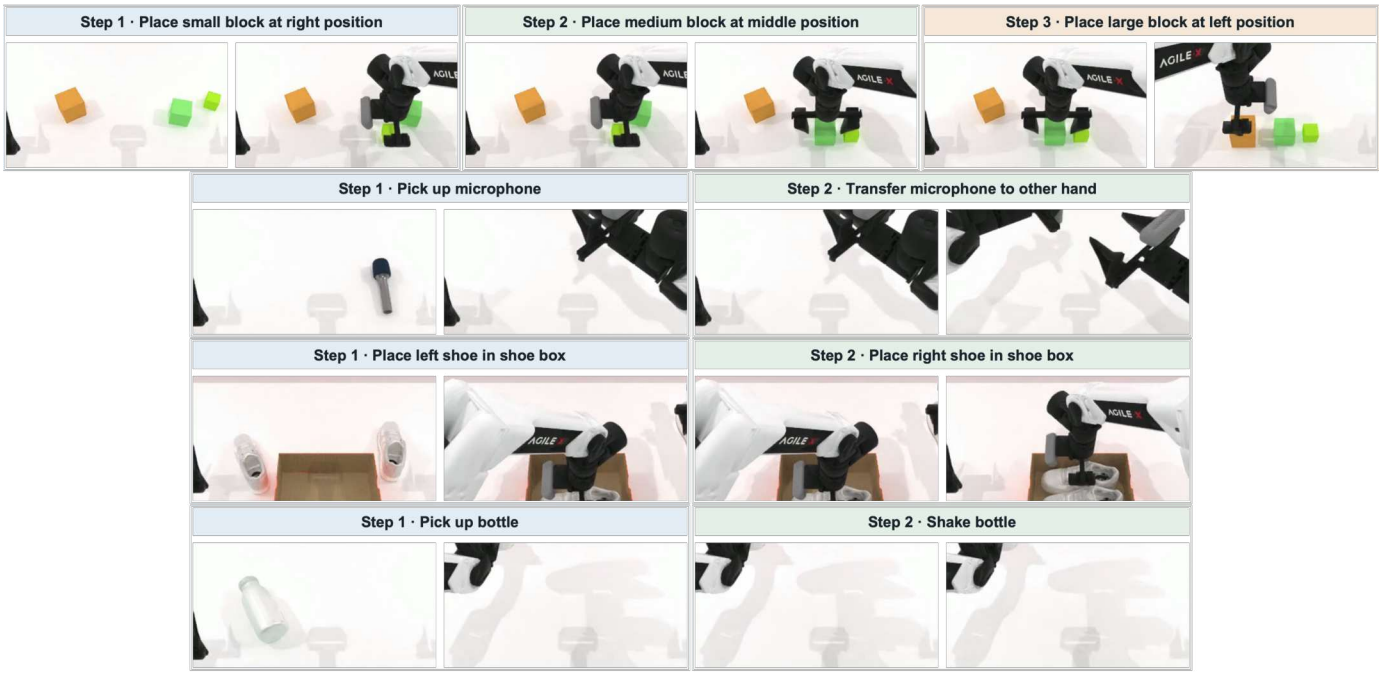


Fig. 12. Additional RoboTwin examples. From top to bottom: Blocks Ranking Size, Handover Mic, Place Dual Shoes, and Shake Bottle. The stage boundaries separate size-dependent placements, an inter-arm transfer, left- and right-shoe placements, and bottle acquisition from shaking.

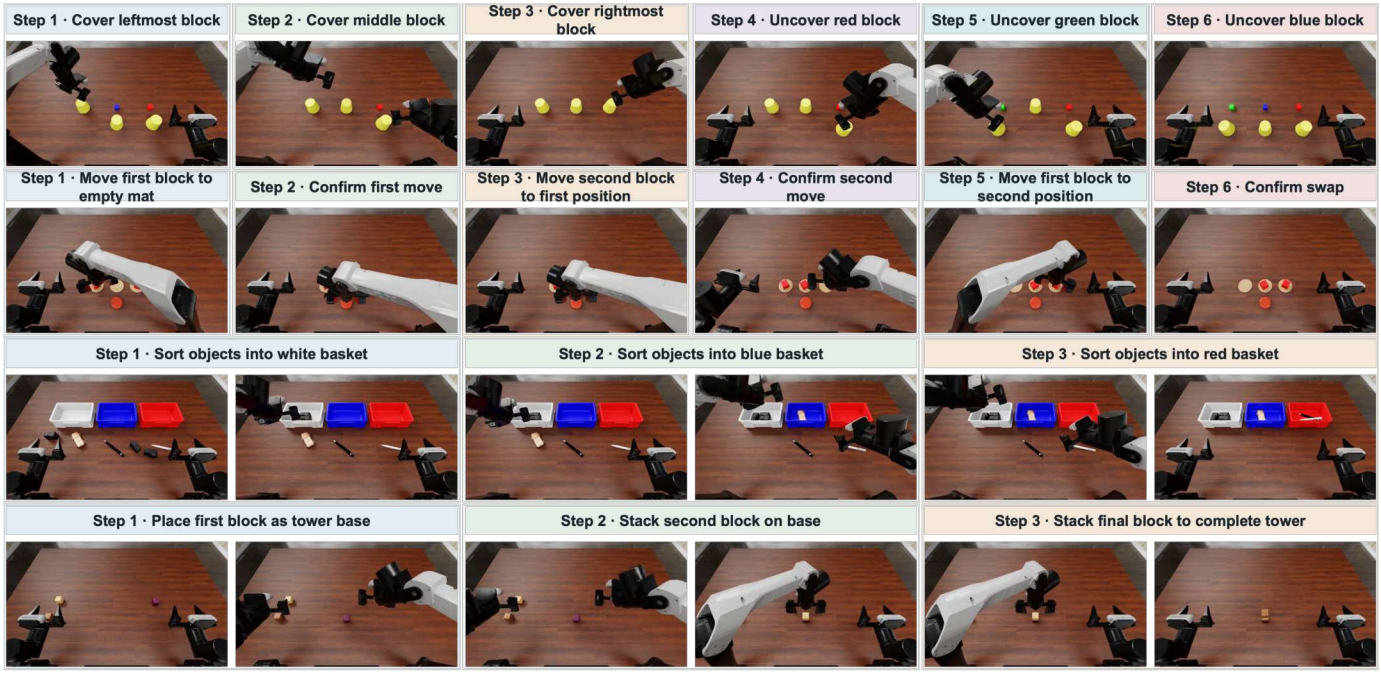


Fig. 13. Semantic subtask segmentation on RoboDojo. From top to bottom: Cover Blocks, Swap Blocks, Classify Objects, and Stack Blocks. Covering and uncovering are represented as distinct stages; in Swap Blocks, each confirmation stage corresponds to a button press following a transfer. Sorting is grouped by the destination basket.

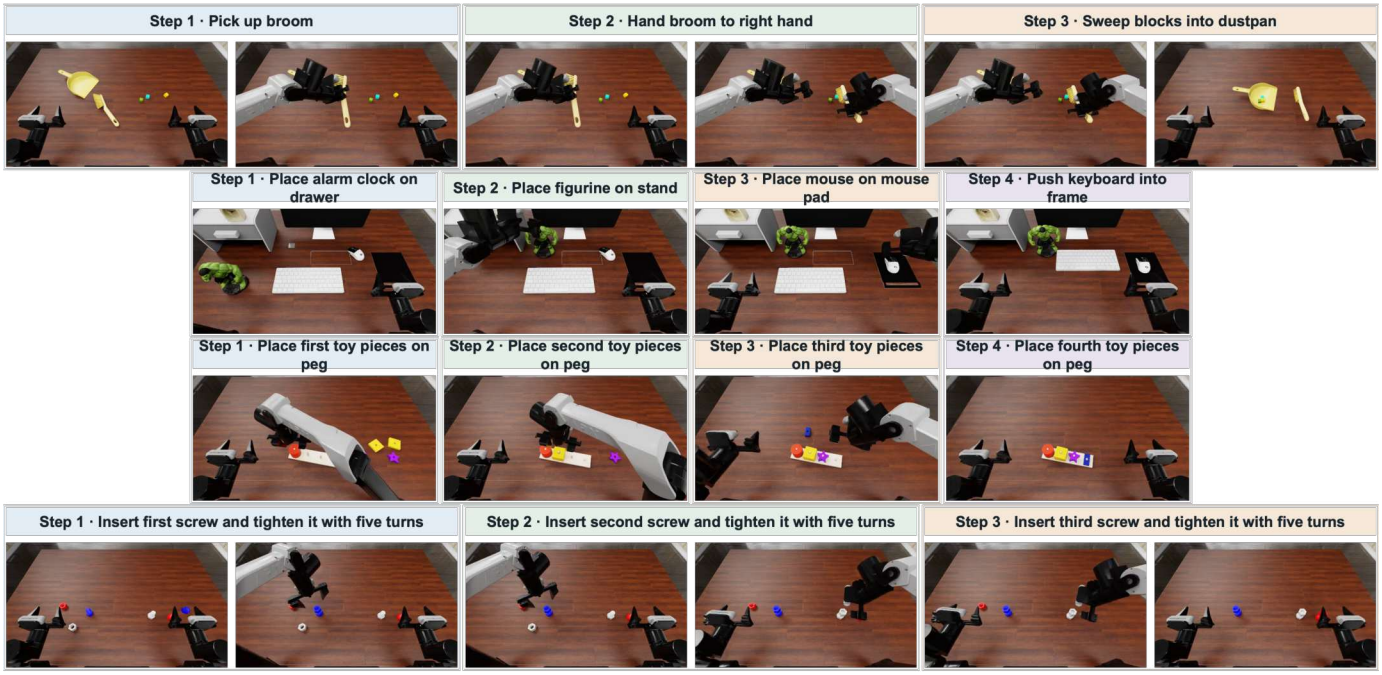


Fig. 14. Additional RoboDojo examples. From top to bottom: Sweep Blocks, Organize Table, Play Stacking Toy, and Fasten Screws. The decompositions preserve tool acquisition and transfer, object-specific placement objectives, and repeated fastening operations. In Fasten Screws, insertion and five tightening turns form one subtask per screw.